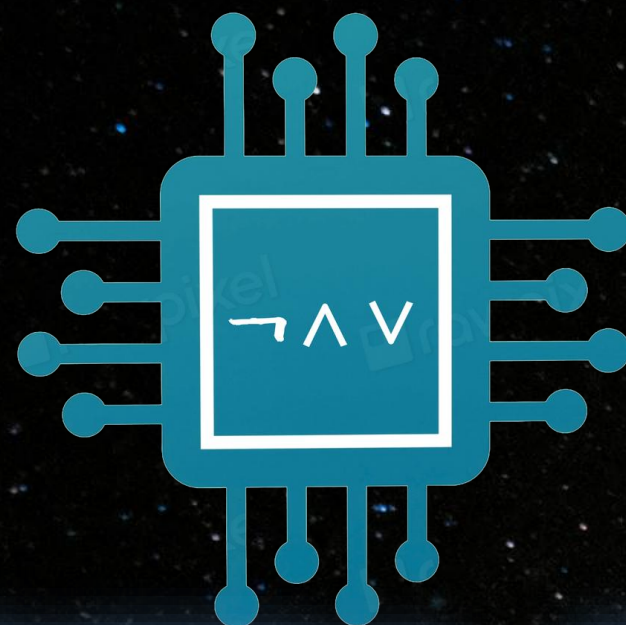
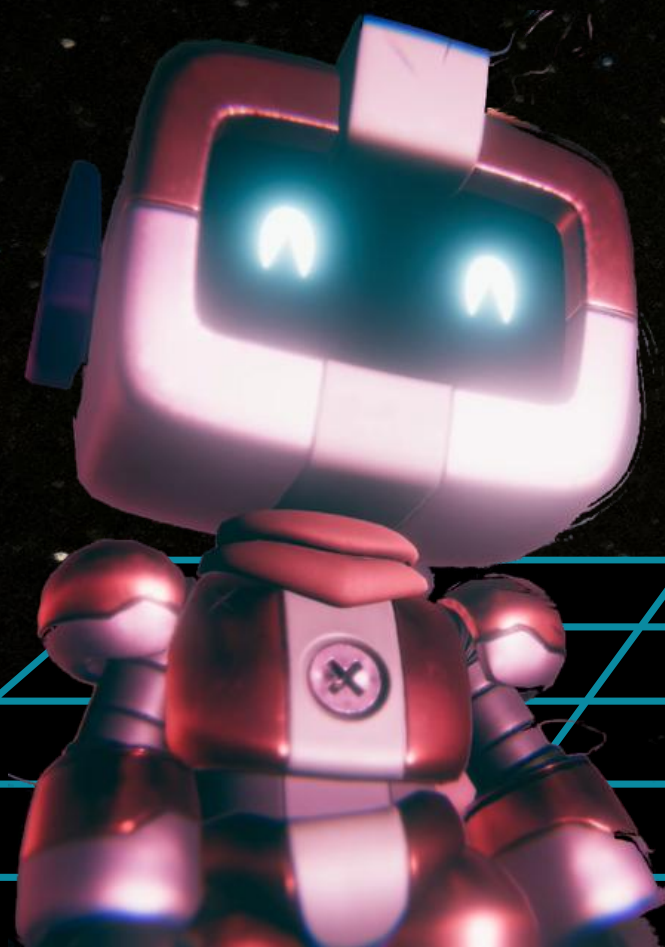




NOT A NODOR

Save your computer from bugs!

START



TOPICS COVERED

GAME OVERVIEW	<i>General explanation of the game</i>
MENUS	<i>Main Menu, Save Menu, Pause Menu and Option settings</i>
GAME SYSTEM AND GATE LOGIC	<i>NOT, AND, OR logic, grabbing system and implementation</i>
CUTSCENES MENAGMENT AND NPC INTERACTION	<i>Timeline and NPC Interactable script</i>
FUTURE EXPANSIONS	<i>Future way to expand the game</i>

TOPIC 1

GAME OVERVIEW

GENERAL EXPLANATION OF THE GAME

GAME OVERVIEW

The game takes place *inside a computer*

A small robot spends his days checking the system status and making sure everything runs smoothly

However, one day, he discovers that the computer has been infected with *dangerous bugs*

Determined to save the system, the robot sets off on a mission to eliminate the bugs and restore the computer to normal

THE GAME LEVELS

The game has a total of 4 level, each of them explores one or more functionalities:

LEVEL 1

NOT

LEVEL 2

AND - ICE

LEVEL 3

OR - BOUNCE

LEVEL 4

MASTER

Each level takes place inside a single room.
In every room, the only exit door is blocked by
a giant green spider.

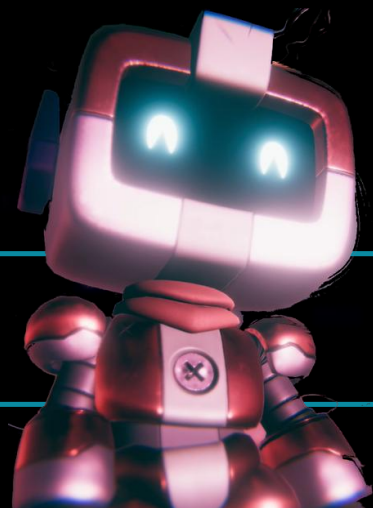
GAME OVERVIEW

The giant spiders are the “bugs” infecting the computer

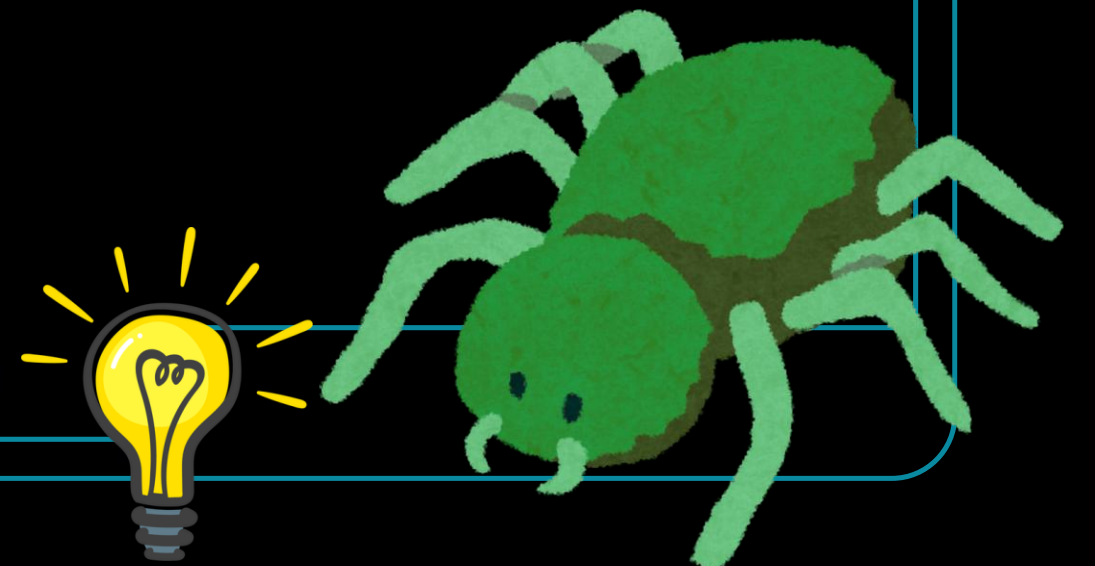
In front of the spider there is a **lamp**

Inside the room different types of blocks and tiles can be found

To move on to the next level the player has to solve a **puzzle** by interacting with said blocks to turn on the lamp in front of the spider



AHH THOSE DAMN
BUGS!



TYPE OF BLOKS

The bloks the player can interact are:

NOT

LOGIC GATE

AND

LOGIC GATE

OR

LOGIC GATE

GREEN EMITTERS

LASER CANNONS

RED EMITTERS

LASER CANNONS

MOVABLE BLOCK

OBSTACLE

- The logic gates replicate the behavior of their counterpart logical operators.
- The emitters release either a **green** or **red** laser when activated, otherwise they emit a gray laser.

TYPE OF TILES

The floor can be made by three different tiles:

NORMAL

AVERAGE FLOOR

ICE

SLIPPERY TILES

BOUNCE

GO BACK TILES

- Normal tiles do not alter the blocks movement.
- Ice tiles make the blocks **slide** in a single direction until they reach the end of the ice path, just like slippery ice.
- Bounce tiles make the blocks **bounce back** to the tile they came from.

TOPIC 2

MENUS

MAIN MENU, SAVE MENU, PAUSE MENU AND
OPTION SETTINGS

THE MENUS

The following types of menus were implemented in the game:

MAIN MENU

Menu Controller Script
Saving Controller Script

SAVE MENU

Saving Controller Script

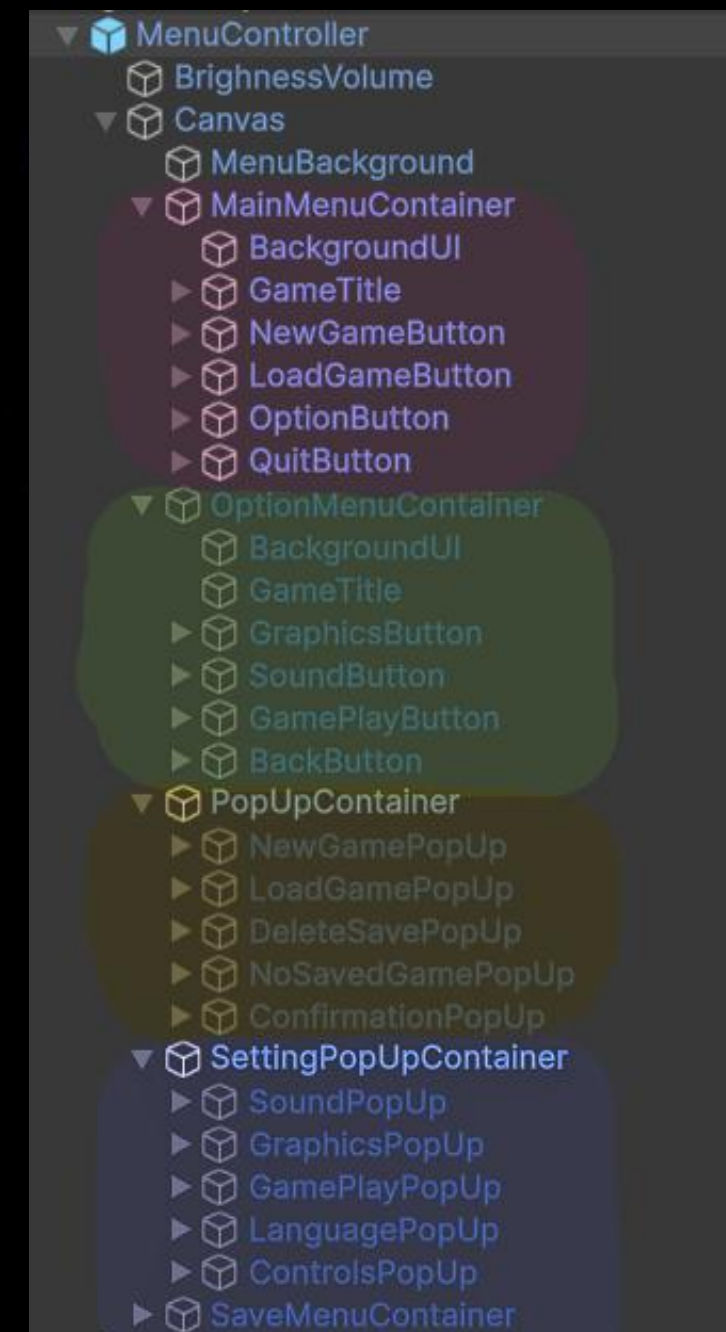
PAUSE MENU

Menu Controller Script
Pause Menu Controller Script

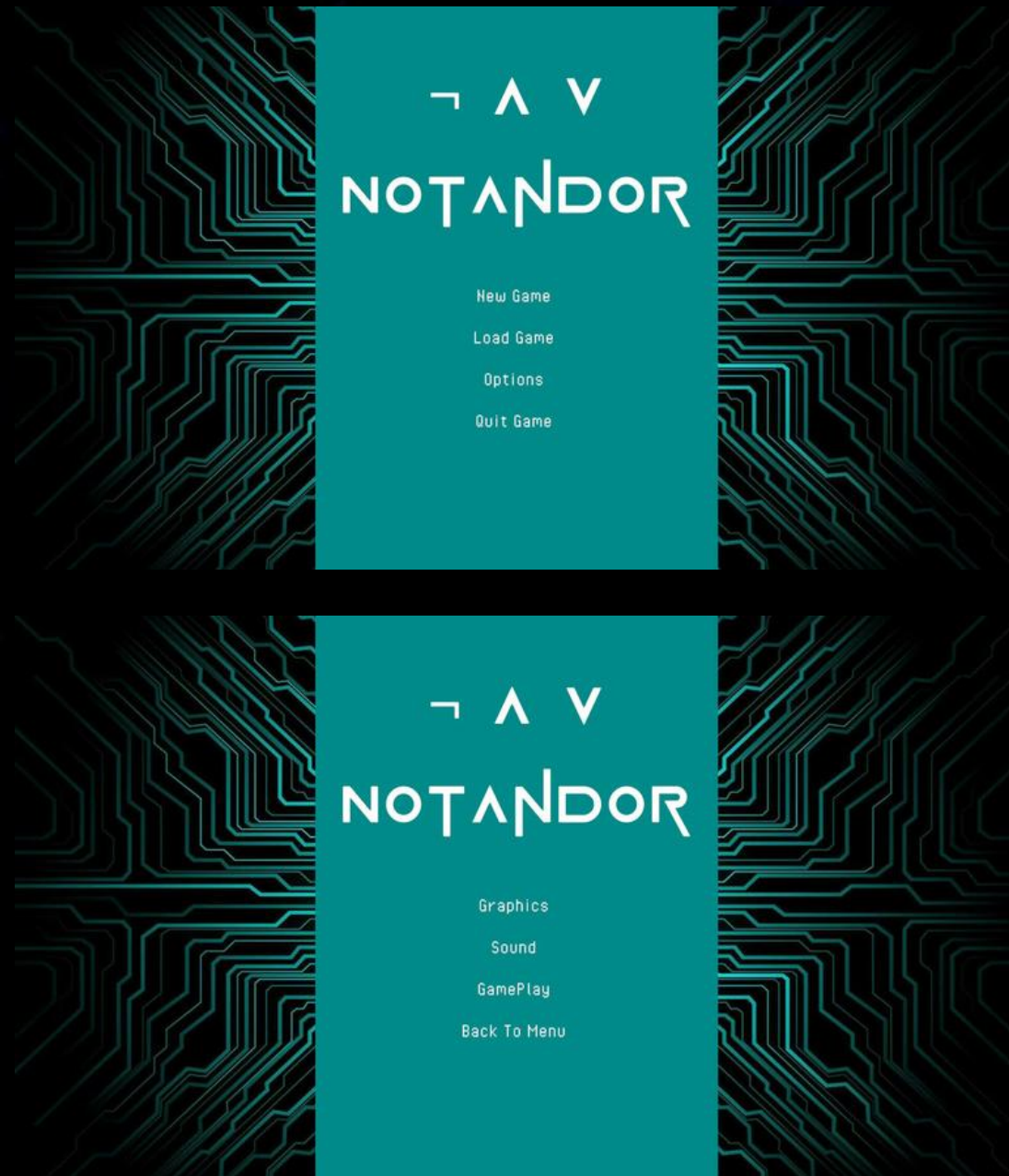
MENU STRUCTURE

All menus in the game share the same basic structure.

Each menu is built using a **Canvas**, with various containers used to organize the buttons, text, and interactive elements within each menu page.



MAIN MENU



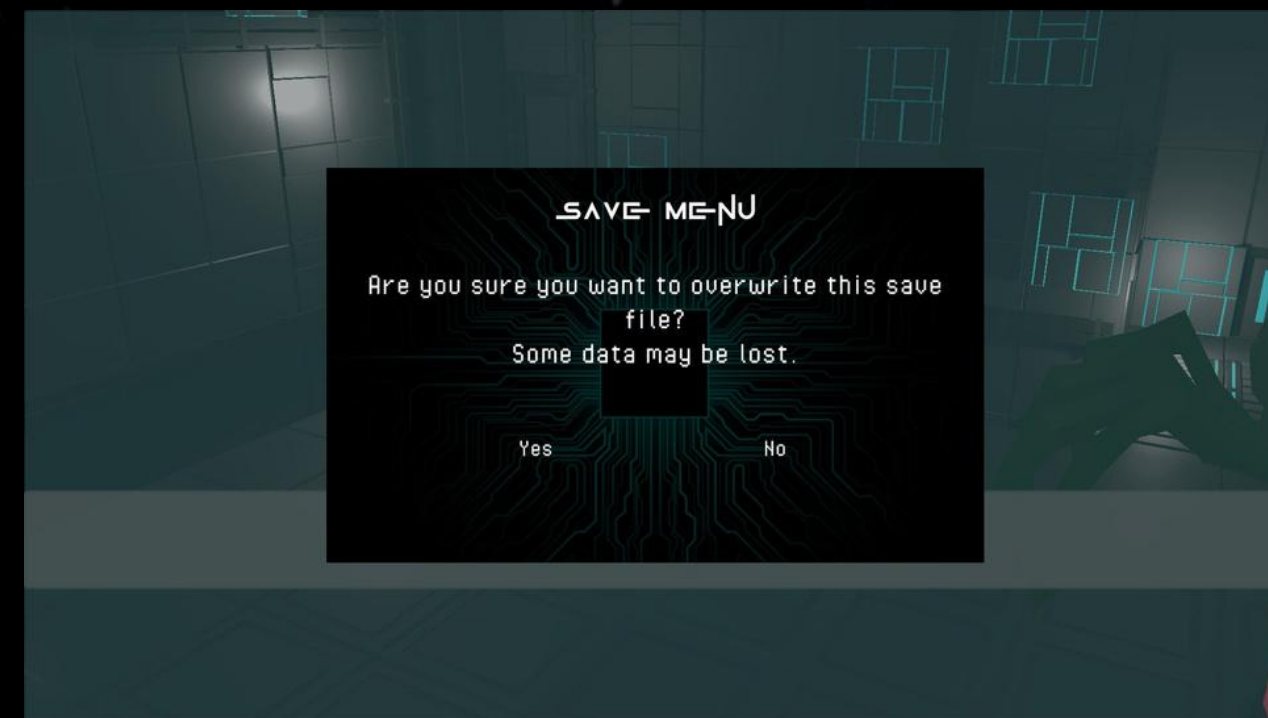
This is the first menu the player sees when launching the game and the last one displayed before quitting.

It allows the player to:

- Start a new game
- Load or delete a previous save
- Change option settings
- Quit the game

CONFIRMATION POP UPS

Before starting a new game, overwriting/deleting a save slot a confirmation pop up will always be shown to make sure this is the intended player action.



OPTION SETTINGS

The options settings are divided into **3 different submenus**, each dedicated to a specific category:

GRAPHICS

Brightness
Quality
Resolution
Fullscreen

AUDIO

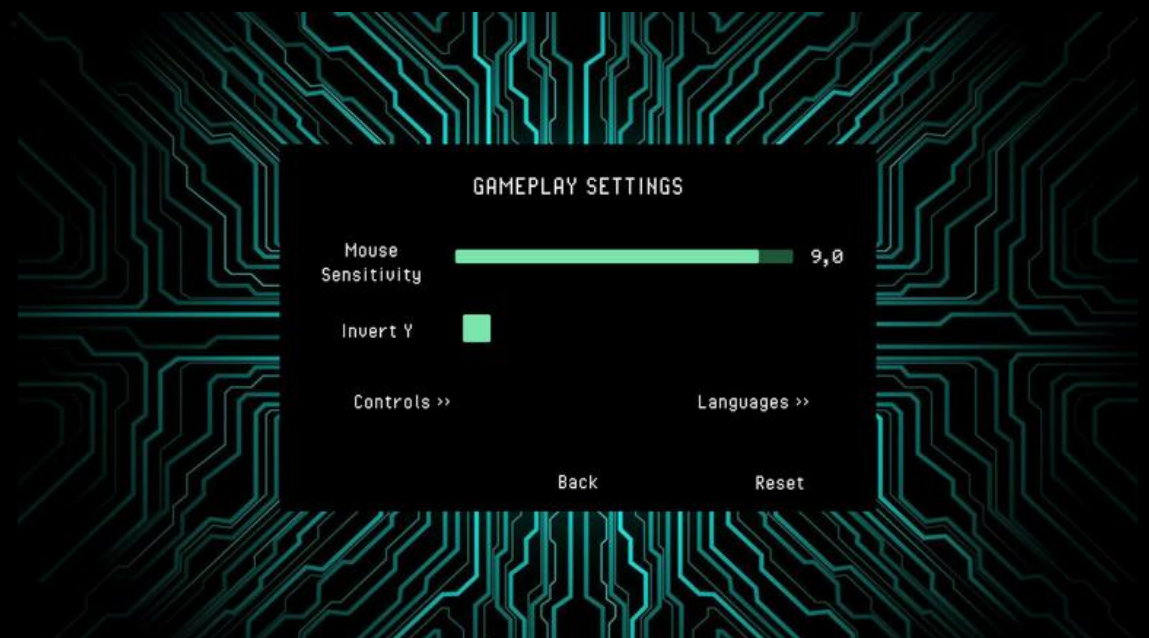
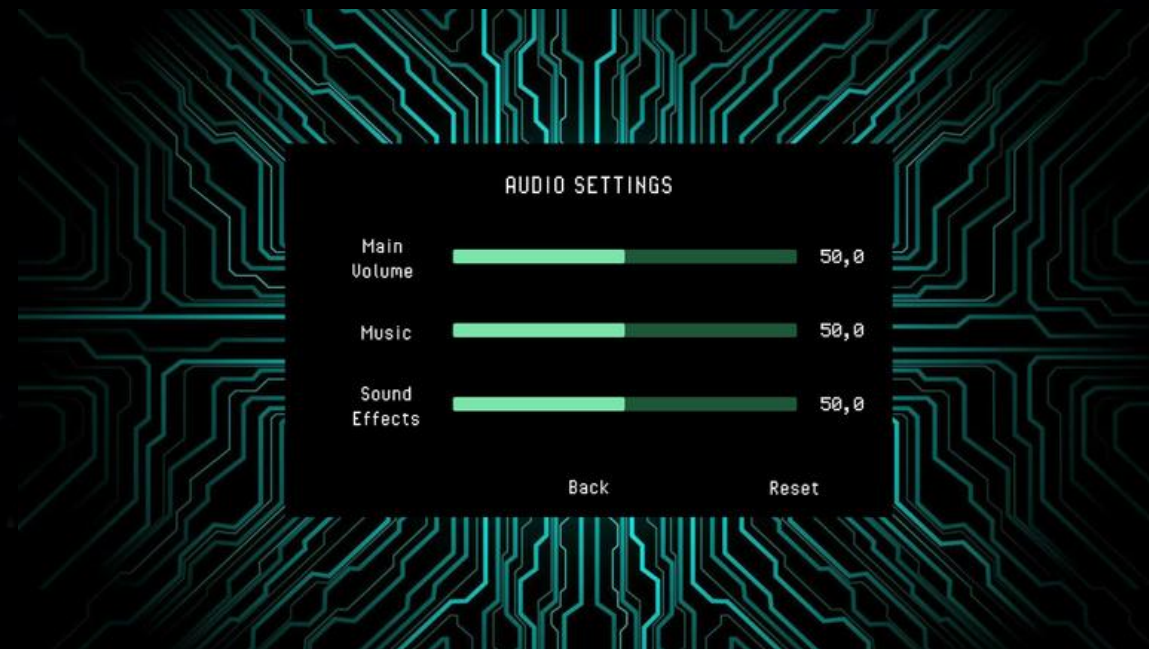
Master volume
Music volume
SFX volume

GAMEPLAY

Mouse Sensitivity
Invert Y
Controls
Language

Each submenu also includes a **reset button**, which restores all settings to their default values.

OPTION SETTINGS



PAUSE MENU

The pause menu can only be accessed during gameplay by pressing the **ESC button**.

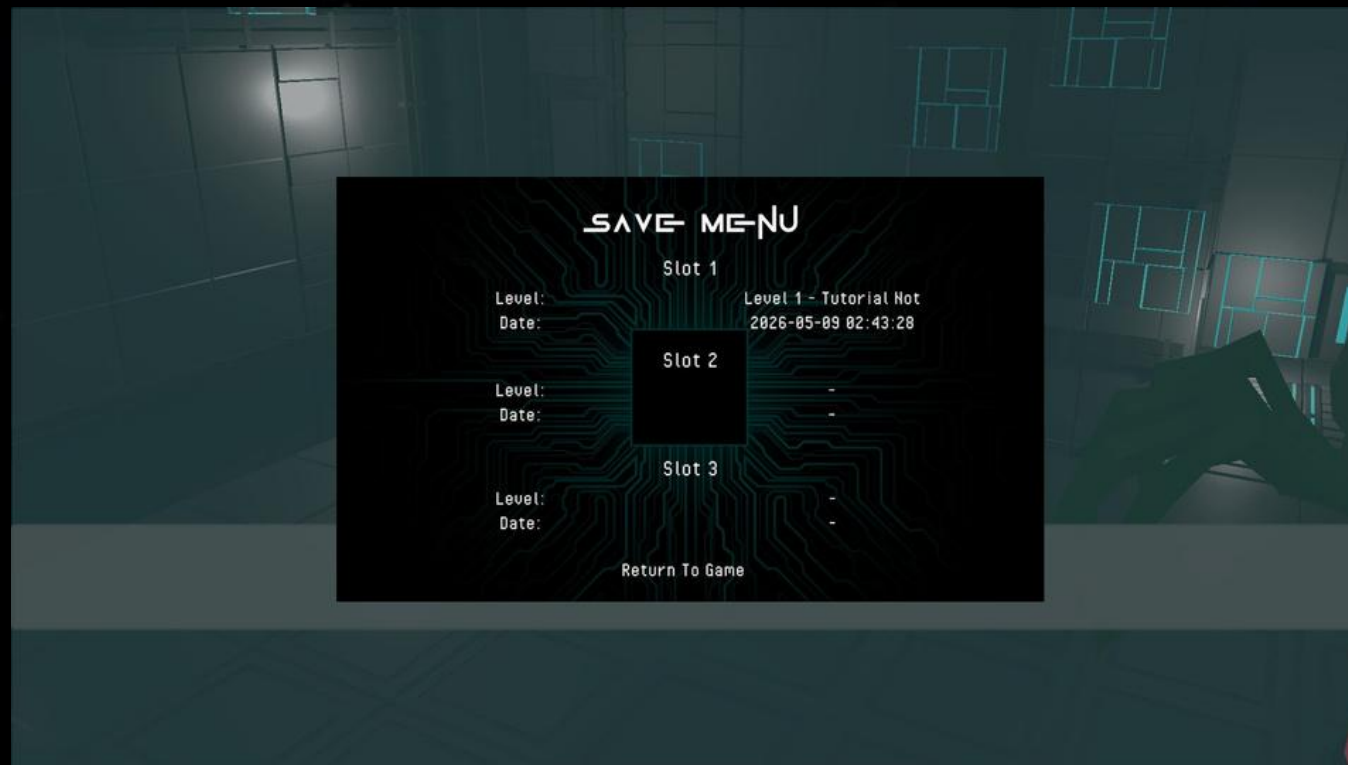
It allows the player to:

- Access the option settings
- **Restart** the current level
- **Unpause** the game



SAVE MENU

This menu can only be accessed during gameplay by interacting with the **saving computer** prefab.



It allow the player to **save** their progress.

SCRIPTS

The menus depend on 3 menu scripts:

**SAVING
CONTROLLER**

**MENU
CONTROLLER**

**PAUSE MENU
CONTROLLER**

MENU CONTROLLER

The MenuController contains all the methods related to the **game settings and options**.

It is based on **4 main methods**:

- **Load** → retrieves saved preferences
- **Apply** → saves current settings
- **Set** → updates values in real time based on UI input
- **Reset** → restores default settings

MENU CONTROLLER

1 reference

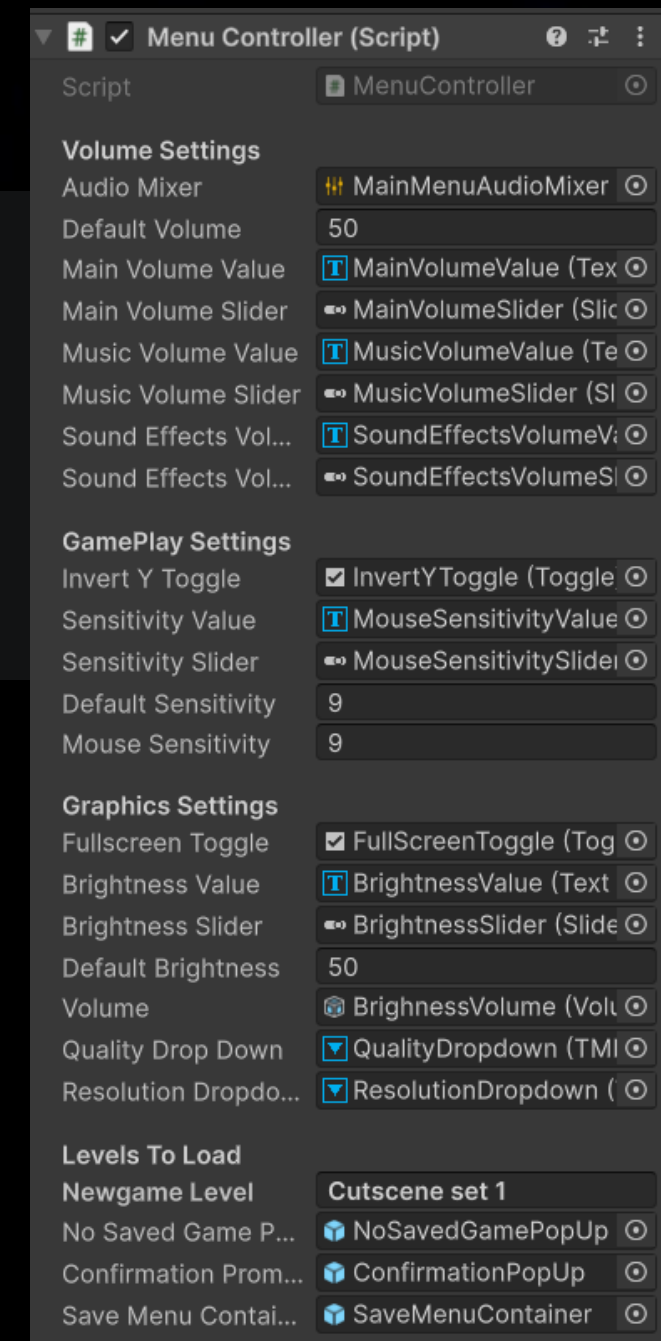
```
private void LoadVolume()
{
    musicVolumeSlider.value = PlayerPrefs.GetFloat("MusicVolume");
    mainVolumeSlider.value = PlayerPrefs.GetFloat("MainVolume");
    soundEffectsVolumeSlider.value = PlayerPrefs.GetFloat("SoundEffectsVolume");
}
```

1 reference

```
public void SetMusicVolume(float volume)
{
    musicVolumeValue.text = volume.ToString("0.0");

    // Convert the linear volume (0-100) to decibels for the audio mixer
    float normalized = volume / 100f;
    normalized = Mathf.Clamp(normalized, 0.0001f, 1f);
    float db = Mathf.Log10(normalized) * 20;

    audioMixer.SetFloat("music", db);
}
```



PAUSE MENU CONTROLLER

The **Pause Menu Controller** script contains all the methods related to **pausing**, **resuming**, and **restarting** the game.

```
public void RestartLevel()
{
    menuController.PlayButtonSoundEffect();
    Time.timeScale = 1f;
    GameState.IsPaused = false;
    SceneManager.LoadScene(SceneManager.GetActiveScene().name);
}
```

```
1 reference
public void Pause()
{
    menuController.PlayButtonSoundEffect();
    pauseMenu.SetActive(true);
    Time.timeScale = 0f;

    GameState.IsPaused = true;

    Cursor.lockState = CursorLockMode.None;
    Cursor.visible = true;
}
```

SAVING CONTROLLER

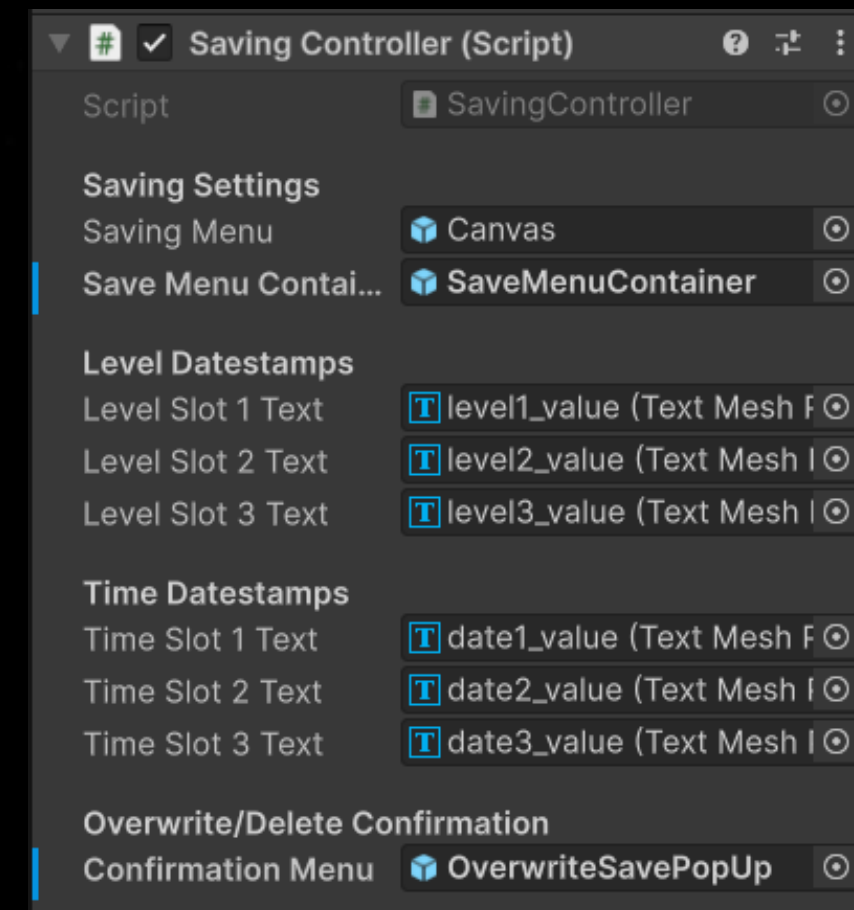
The **Saving Controller** contains all the methods related to **creating, overwriting, loading, and deleting** save slots.

```
public void SaveToSlot(int slot)
{
    string currentLevel = SceneManager.GetActiveScene().name;
    string currentTime = System.DateTime.Now.ToString("yyyy-MM-dd HH:mm:ss");

    PlayerPrefs.SetString($"SaveSlot{slot}_Level", currentLevel);
    PlayerPrefs.SetString($"SaveSlot{slot}_Time", currentTime);

    LoadValuesFromSlot();
}
```

```
public void Deleteslot(int slot)
{
    PlayerPrefs.DeleteKey($"SaveSlot{slot}_Level");
    PlayerPrefs.DeleteKey($"SaveSlot{slot}_Time");
    LoadValuesFromSlot();
}
```



GAME STATE SCRIPT

The script uses **static global boolean variables** to keep track of the **different states of the game**.

Its main purpose is to ensure that **multiple menus cannot be opened simultaneously**.

```
1  using UnityEngine;
2
3  0 references
4  public static class GameState
5  {
6      0 references
7      public static bool IsPaused;
8      0 references
9      public static bool IsSaveMenuOpen;
10 }
```

TOPIC 3

GAME SYSTEM AND GATE LOGIC

NOT, AND, OR LOGIC, GRABBING SYSTEM AND
IMPLEMENTATION

GAME SYSTEMS

There are 4 principal **game systems**:

GRABBING SYSTEM

GRID SYSTEM

**INTERACTING
SYSTEM**

LASER SYSTEM

GAME SYSTEMS

Various custom layers have been created and used:

- Interactable
- Movable
- Grabbed
- GrabbedInert
- Laser
- Tile
- Player

INTERACTIONS

The **Player** can **interact** with various objects and each of them can implement **differeⁿts behaviours** and show different messages on the screen.

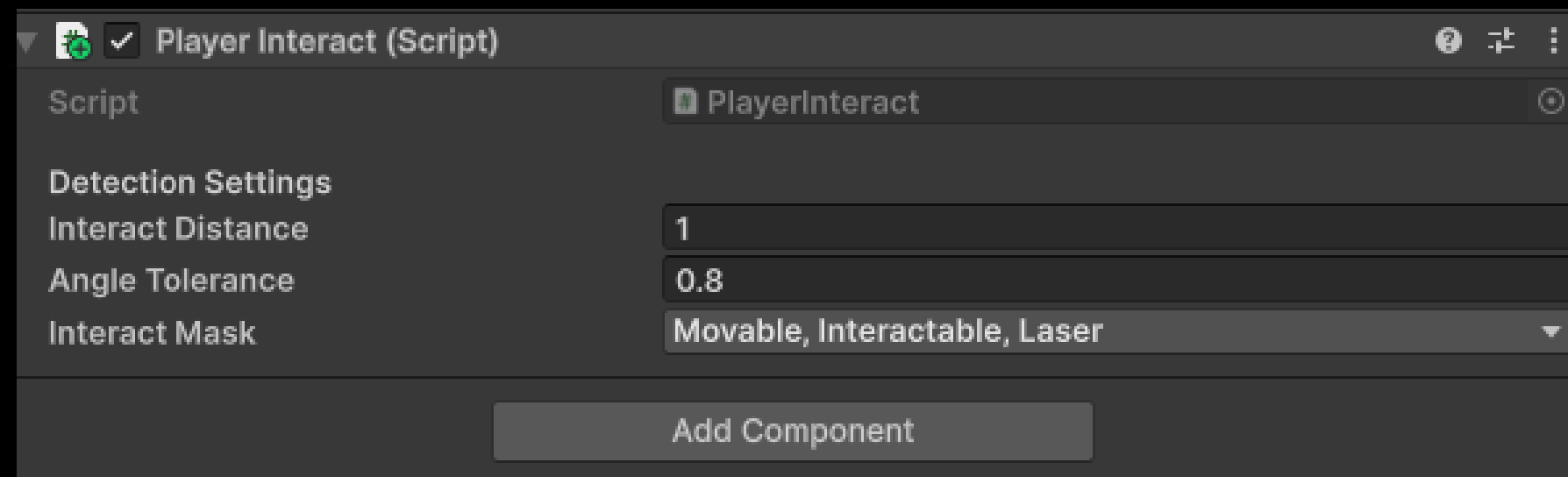
INTERACTIONS

This system uses **collisions** and **scalar products** to find the best object to interact with as the **raycast** method is less reliable due to the third person view of the game.

```
void DetectInteractable()
{
    Vector3 origin = transform.TransformPoint(controller.center);
    Collider[] hits = Physics.OverlapSphere(origin, interactDistance, interactMask);
    Interactable bestInteractable = null;
    float bestDot = angleTolerance;
    foreach (Collider col in hits)
    {
        Interactable interactable = col.GetComponent<Interactable>();
        if (interactable == null || !interactable.canInteract)
            continue;
        Vector3 closestPoint = col.ClosestPoint(origin);
        Vector3 dirToTarget = (closestPoint - origin).normalized;
        float dot = Vector3.Dot(transform.forward, dirToTarget);
        if (dot > bestDot)
        {
            bestDot = dot;
            bestInteractable = interactable;
        }
    }
    if (bestInteractable != currentInteractable)
    {
        currentInteractable?.OnExitRange();
        currentInteractable = bestInteractable;
        currentInteractable?.OnEnterRange();
    }
    if (currentInteractable != null && !currentInteractable.canInteract)
    {
        currentInteractable.OnExitRange();
        currentInteractable = null;
    }
}
```

INTERACTIONS

There are various parameters to customize the range and precision of the **interactions**.



INTERACTIONS

```
public virtual void OnEnterRange()
{
    if (!canInteract) return;
    playerInRange = true;
    if (isNPC)
        UIManager.Instance.ShowDialog(promptMessage);
    else
        UIManager.Instance.ShowPrompt(promptMessage);
}
3 riferimenti
public virtual void OnExitRange()
{
    playerInRange = false;
    UIManager.Instance.HidePrompt();
    UIManager.Instance.HideDialog();
}
1 riferimento
public void TryInteract()
{
    if (playerInRange && canInteract)
        Interact();
}
6 riferimenti
protected abstract void Interact();
```

Each object controls
what to do when the
Player is interacting
with it.

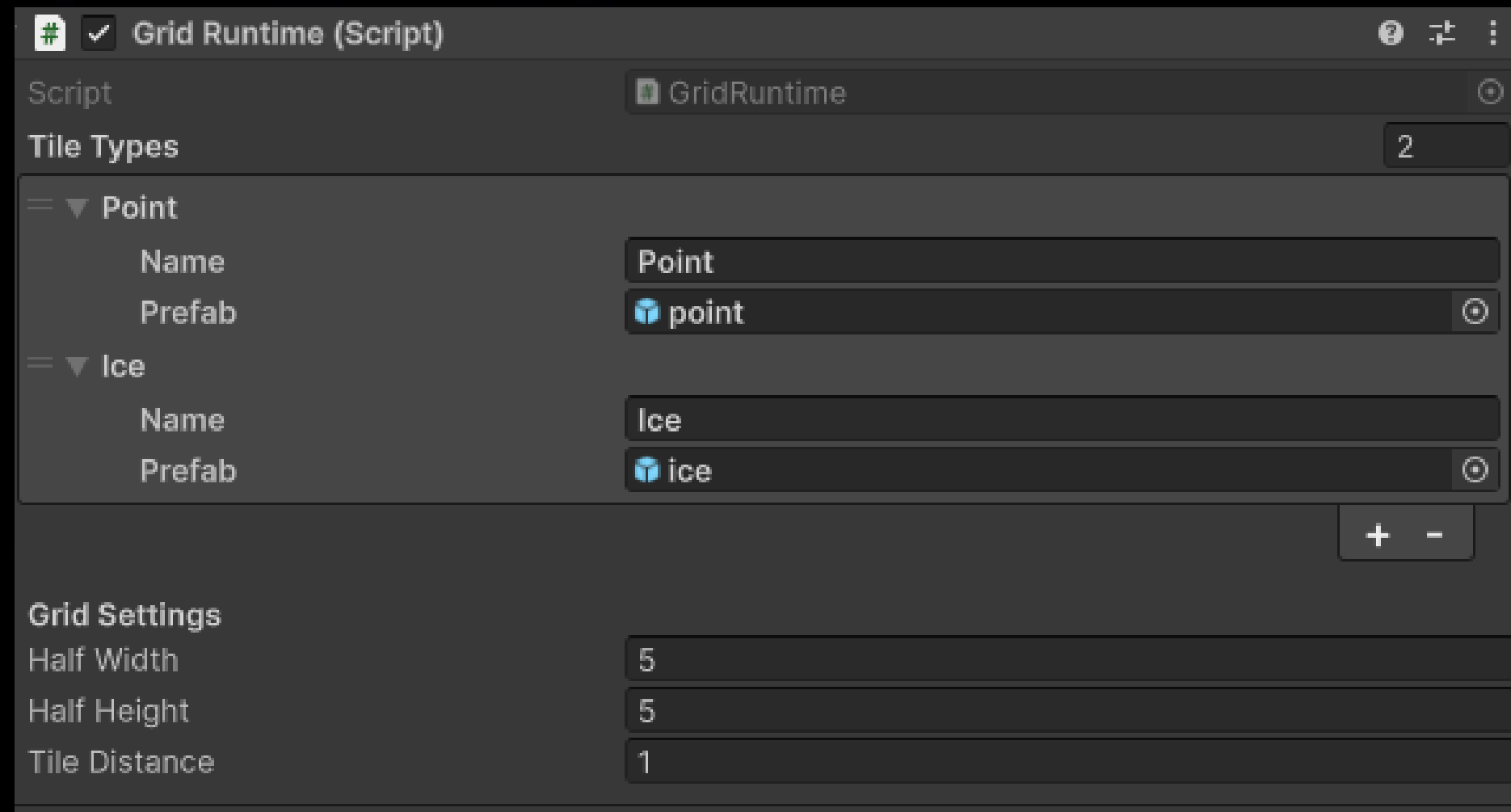
GRID

The **grid** is maintained by an object and is a **matrix** of objects instantiated from the correct **prefab** depending on the type of the wanted tile.



GRID

The **grid** can be customized in **size**, **distance** between the tiles and its **composition**.



TYPE OF TILES



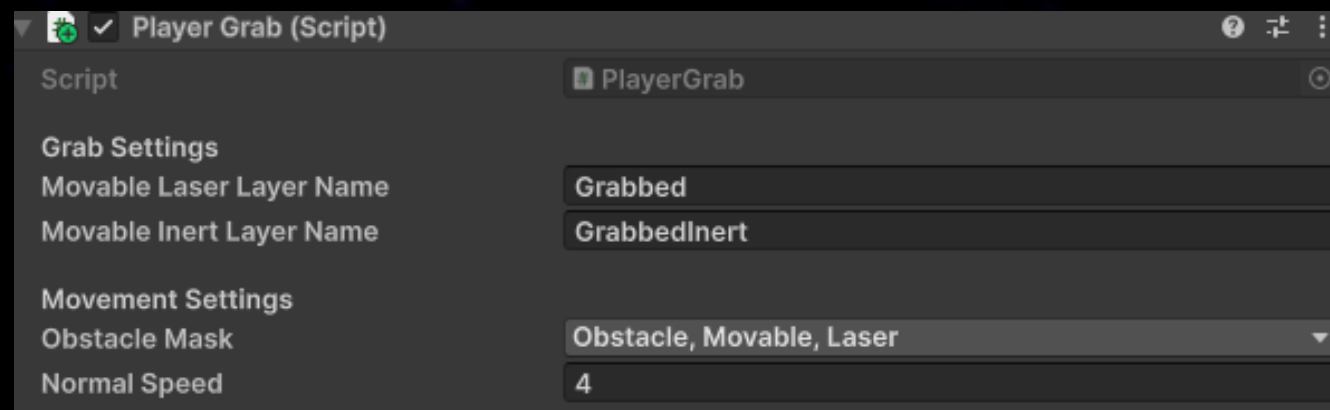
There are `three types` of `tiles` and each decides if the `player` can continue the `movement` and how it should change after passing over it.

TYPE OF TILES

```
public virtual Vector3 ModifyMovement(Vector3 currentPos, ref Vector3 direction, float gridSize, LayerMask obstacleMask)
{
    return currentPos + direction * gridSize;
}
```

It returns the **next position** after evaluating the **tile logic**.

GRABBING



This mechanic
relies on the **grid**
and the ability to
interact with the
objects.

```
//[RequireComponent(typeof(PlayerInteract))]
//[RequireComponent(typeof(CharacterController))]
Messaggio Unity | 0 riferimenti
void Start()
{
    playerInteract = GetComponent<PlayerInteract>();
    controller = GetComponent<CharacterController>();
    moveScript = GetComponent<MovementInput>();
    grid = GameObject.FindObjectOfType<GridRuntime>();
    if (grid == null)
    {
        Debug.LogError("PlayerGrab: GridRuntime non trovato! Disattivo PlayerGrab.");
        enabled = false;
        return;
    }
    if (moveScript != null)
    {
        originalVelocity = moveScript.Velocity;
        originalAllowRotation = moveScript.allowPlayerRotation;
    }
}
```

GRABBING

When the **player** grabs an object they are **locked** together and the player is **put in front** of the object while **looking** at it.

```
public void StartGrab(MovableObject obj)
{
    if (isGrabbing)
    {
        Debug.LogWarning("Tentativo di afferrare ridondante.");
        return;
    }
    grabbedObject = obj;
    Vector3 objPos = Snap(obj.transform.position);
    grabbedObject.transform.position = objPos;
    Vector3 snappedPlayerPos = Snap(transform.position);
    Vector3 directionFromBlockToPlayer = snappedPlayerPos - objPos;
    directionFromBlockToPlayer.y = 0f;
    if (Mathf.Abs(directionFromBlockToPlayer.x) > Mathf.Abs(directionFromBlockToPlayer.z))
    {
        grabDirection = new Vector3(Mathf.Sign(directionFromBlockToPlayer.x), 0f, 0f);
    }
    else
    {
        grabDirection = new Vector3(0f, 0f, Mathf.Sign(directionFromBlockToPlayer.z));
    }
    Vector3 targetPlayerPos = objPos + grabDirection * grid.tileDistance;
    targetPlayerPos.y = transform.position.y;
    if (controller != null)
        controller.enabled = false;
    transform.position = targetPlayerPos;
    Vector3 lookDir = objPos - transform.position;
    lookDir.y = 0f;
    if (lookDir.sqrMagnitude > 0.001f)
    {
        transform.rotation = Quaternion.LookRotation(lookDir.normalized);
    }
    if (controller != null)
        controller.enabled = true;
    grabDirection = -grabDirection;
    if (moveScript != null)
    {
        moveScript.Velocity = 0;
        moveScript.allowPlayerRotation = 999f;
    }
    isGrabbing = true;
}
```

GRABBING

It changes the **layer** of the **grabbed** object so it doesn't interfere with the **player** movement and makes the **grid** visible.

```
isGrabbing = true;
originalLayer = obj.gameObject.layer;
string targetLayer = obj.GetComponentInChildren<LaserNode>() != null
    ? movableLaserLayerName
    : movableInertLayerName;
SetLayerRecursively(obj.gameObject, LayerMask.NameToLayer(targetLayer));
Debug.Log($"<color=green>Grab iniziato su {obj.name}. Layer assegnato: {targetLayer}</color>");
if (playerInteract != null)
{
    playerInteract.SetInteracting(true);
    playerInteract.ClearCurrentState();
}
if (grid != null)
{
    grid.SetGridVisible(true);
    grid.SetGridToggleable(false);
}
else
{
    Debug.LogError("PlayerGrab: GridRuntime instance not found! Assicurati che ci sia un oggetto con GridRuntime nella scena.");
}
}
```

GRABBING

```
public void StopGrab()
{
    if (grabbedObject != null)
    {
        SetLayerRecursively(grabbedObject.gameObject, originalLayer);
        Debug.Log($"<color=red>Grab terminato. Layer ripristinato a {originalLayer}</color>");
    }
    grabbedObject = null;
    if (playerInteract != null) playerInteract.SetInteracting(false);
    if (moveScript != null)
    {
        moveScript.Velocity = originalVelocity;
        moveScript.allowPlayerRotation = originalAllowRotation;
    }
    if (grid != null)
    {
        grid.SetGridVisible(false);
        grid.SetGridTogglable(true);
    }
    else
    {
        Debug.LogError("PlayerGrab: GridRuntime non trovato! Assicurati che ci sia un oggetto con GridRuntime nella scena.");
    }
    isGrabbing = false;
}
```

When the **grab** stops it restores the **original** state of the object, player and the grid.

GRABBING

The player can **rotate** and **move** the objects along the **grid** while checking for **obstacles** or **unfinished** previous movements.

```
void HandleMovement(Vector3 dir)
{
    if (isMoving)
    {
        Debug.Log("<color=grey>Input ignorato: movimento in corso</color>");
        return;
    }
    if (isRotating)
    {
        Debug.Log("<color=grey>Input ignorato: rotazione in corso</color>");
        return;
    }
    if (grabbedObject == null)
    {
        Debug.LogWarning("Tentativo di muovere senza oggetto grab.");
        return;
    }
    Vector3 startBlock = Snap(grabbedObject.transform.position);
    Vector3 currentBlock = startBlock;
    List<Vector3> blockPath = new List<Vector3>();
    blockPath.Add(currentBlock);
    int maxSteps = 50;
    Debug.Log($"<color=white>START MOVE dir: {dir}</color>");
    for (int i = 0; i < maxSteps; i++)
    {
        GridTileBase currentTile = GetTileAt(currentBlock);
        if (currentTile == null)
        {
            Debug.Log("<color=red>Tile corrente NULL</color>");
            break;
        }
        Debug.Log($"<color=yellow>STEP {i} | Tile: {currentTile.name}</color>");
        Vector3 nextBlock = currentTile.ModifyMovement(
            currentBlock,
            ref dir,
            grid.tileDistance,
            obstacleMask
        );
        if (nextBlock == currentBlock)
        {
            Debug.Log("<color=orange>Blocco fermato</color>");
            break;
        }
        Vector3 nextPlayer = nextBlock - grabDirection * grid.tileDistance;
        bool blockInside = GetTileAt(nextBlock) != null;
        Debug.Log($"BlockInside: {blockInside}");
        if (!blockInside)
        {
            Debug.Log("<color=red>Blocco fuori griglia! Movimento interrotto.</color>");
            break;
        }
        Collider blockCol = grabbedObject.GetComponent<Collider>();
        Vector3 blockCenter = nextBlock + blockCol.bounds.center - grabbedObject.transform.position;
    }
}
```

GRABBING

After finding the **next position** it's checked for collisions.

```
Collider blockCol = grabbedObject.GetComponent<Collider>();
Vector3 blockCenter = nextBlock + blockCol.bounds.center - grabbedObject.transform.position;
Vector3 blockHalfExtents = blockCol.bounds.extents * 0.9f;
Collider[] blockHits = Physics.OverlapBox(blockCenter, blockHalfExtents, Quaternion.identity, obstacleMask);
if (blockHits.Length > 0)
{
    Debug.Log("<color=red>Blocco bloccato da:</color>");
    foreach (var h in blockHits)
        Debug.Log($" - {h.name}");
    break;
}
```

GRABBING

```
if (blockHits.Length > 0)
{
    Debug.Log("<color=red>Blocco bloccato da:</color>");
    foreach (var h in blockHits)
        Debug.Log($" - {h.name}");
    break;
}

float halfHeight = controller.height / 2f;
Vector3 checkCenter = nextPlayer + Vector3.up * halfHeight;
Vector3 checkHalfExtents =
    new Vector3(
        controller.radius,
        halfHeight,
        controller.radius
    ) * 0.8f;
Collider[] hits = Physics.OverlapBox(
    checkCenter,
    checkHalfExtents,
    Quaternion.identity,
    obstacleMask
);
if (hits.Length > 0)
{
    foreach (var h in hits)
        Debug.Log($" - {h.name} | Layer: {LayerMask.LayerToName(h.gameObject.layer)}");
    break;
}

currentBlock = nextBlock;
blockPath.Add(currentBlock);
GridTileBase destTile = GetTileAt(currentBlock);
if (destTile == null)
{
    Debug.Log("<color=red>Tile destinazione NULL</color>");
    break;
}

Debug.Log($"<color=cyan>Arrivato su: {destTile.name}</color>");
if (!destTile.ShouldContinue())
{
    Debug.Log("<color=grey>Stop: tile normale</color>");
    break;
}
}

if (blockPath.Count > 1)
{
    isMoving = true;
    StartCoroutine(MoveStepByStep(blockPath));
}
else
{
    Debug.Log("<color=grey>Nessun movimento</color>");
}
```

After creating the eventual
movement path it's executed
and animated if it's not just
the starting position.

GRABBING

Uses **Lerp** for smooth movement.

```
IEnumerator MoveStepByStep(List<Vector3> blockPath)
{
    Debug.Log("<color=white>START ANIMATION</color>");
    for (int i = 1; i < blockPath.Count; i++)
    {
        float baseSpeed = normalSpeed;
        Vector3 startBlock = blockPath[i - 1];
        Vector3 endBlock = blockPath[i];
        GridTileBase tile = GetTileAt(endBlock);
        float speed = baseSpeed;
        if (tile != null)
            speed = tile.GetMoveSpeed(baseSpeed);
        float distance = Vector3.Distance(startBlock, endBlock);
        float duration = distance / speed;
        float completamentoMovimento = 0f;
        Debug.Log($"<color=green>Step {i} | Speed: {speed} | Duration: {duration}</color>");
        while (completamentoMovimento < 1f)
        {
            completamentoMovimento += Time.deltaTime / duration;
            grabbedObject.transform.position =
                Vector3.Lerp(
                    startBlock,
                    endBlock,
                    completamentoMovimento
                );
            transform.position = grabbedObject.transform.position - grabDirection * grid.tileDistance;
            yield return null;
        }
        grabbedObject.transform.position = endBlock;
        transform.position = endBlock - grabDirection * grid.tileDistance;
    }
    isMoving = false;
    Debug.Log("<color=white>END ANIMATION</color>");
}
```

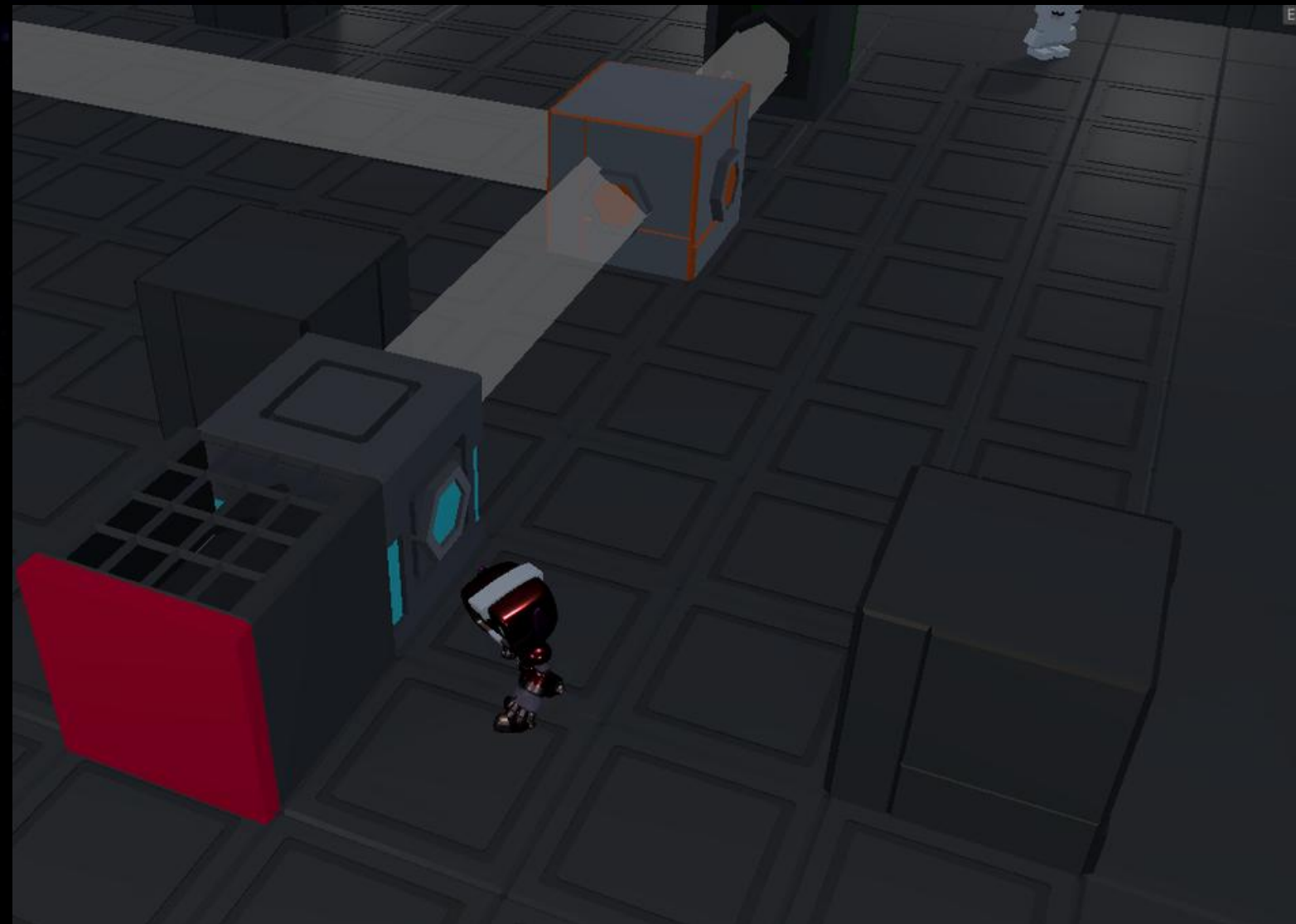
GRABBING

```
void Update()
{
    if (grabbedObject == null)
    {
        isGrabbing = false;
        return;
    }
    if (Input.GetKeyDown(KeyCode.Q) && !isMoving && !isRotating) StopGrab();
    if (!isMoving && !isRotating)
    {
        if (Input.GetKeyDown(KeyCode.R) && grabbedObject != null && grabbedObject.isRotatable) StartCoroutine(RotateObject());
        if (grabbedObject == null || !grabbedObject.isMovable)
        {
            return;
        }
        Vector3 dir = GetInputDirection();
        if (moveScript != null && moveScript.anim != null)
        {
            float targetSpeed = (dir != Vector3.zero) ? 0.5f : 0f;
            moveScript.anim.SetFloat("Blend", targetSpeed, 0.1f, Time.deltaTime);
        }
        if (dir != Vector3.zero)
        {
            Debug.Log($"<color=cyan>--- Tasto premuto! Direzione: {dir} ---</color>");
            HandleMovement(dir);
        }
    }
}
```

If the player is **grabbing** an object it cannot **interact** with other objects. While it's **rotating** or **moving** an object further inputs are **ignored**.

LASER

This is the heart of the game together with the **grabbing** system. It allows the **player** to combine various **logic ports** and **reflecting blocks** to propagate the **laser** and activate the **sensor**.



LASER

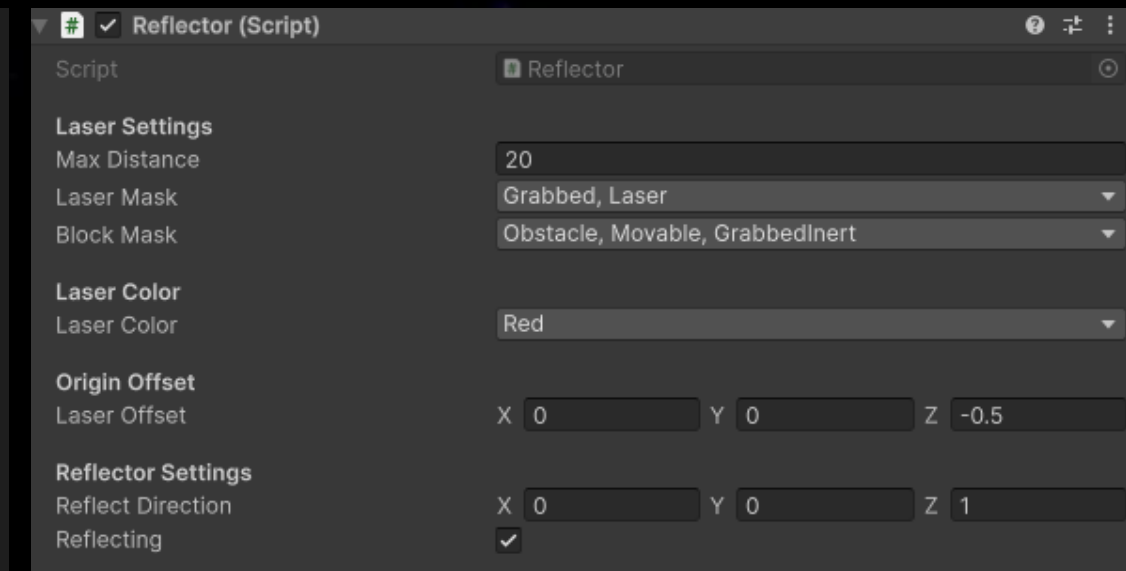
The **LaserNode** script controls the **laser** creation.

```
protected void FireRay(Vector3 origin, Vector3 direction, LaserColor color)
{
    Illuminate(color != LaserColor.Preview, color);
    Vector3 physicsOrigin = origin;
    Vector3 endPoint = physicsOrigin + direction * maxDistance;
    LayerMask combinedMask = laserMask | blockMask;
    RaycastHit hit;
    if (Physics.Raycast(physicsOrigin, direction, out hit, maxDistance, combinedMask))
    {
        endPoint = hit.point;
        int layer = hit.collider.gameObject.layer;
        if (((1 << layer) & laserMask) != 0)
        {
            ILaserReceiver receiver = hit.collider.GetComponent<ILaserReceiver>();
            if (receiver != null)
            {
                receiver.OnLaserHit(hit.point, color);
            }
        }
    }
    DrawLaser(origin, endPoint, color);
}
```

LASER

```
10 riferimenti
public virtual void OnLaserHit(Vector3 hitPoint, LaserColor incomingColor)
{
    isHitThisFrame = true;
    currentIncomingColor = incomingColor;
    currentEmitDirection = CalculateOutputDirection(hitPoint);
}

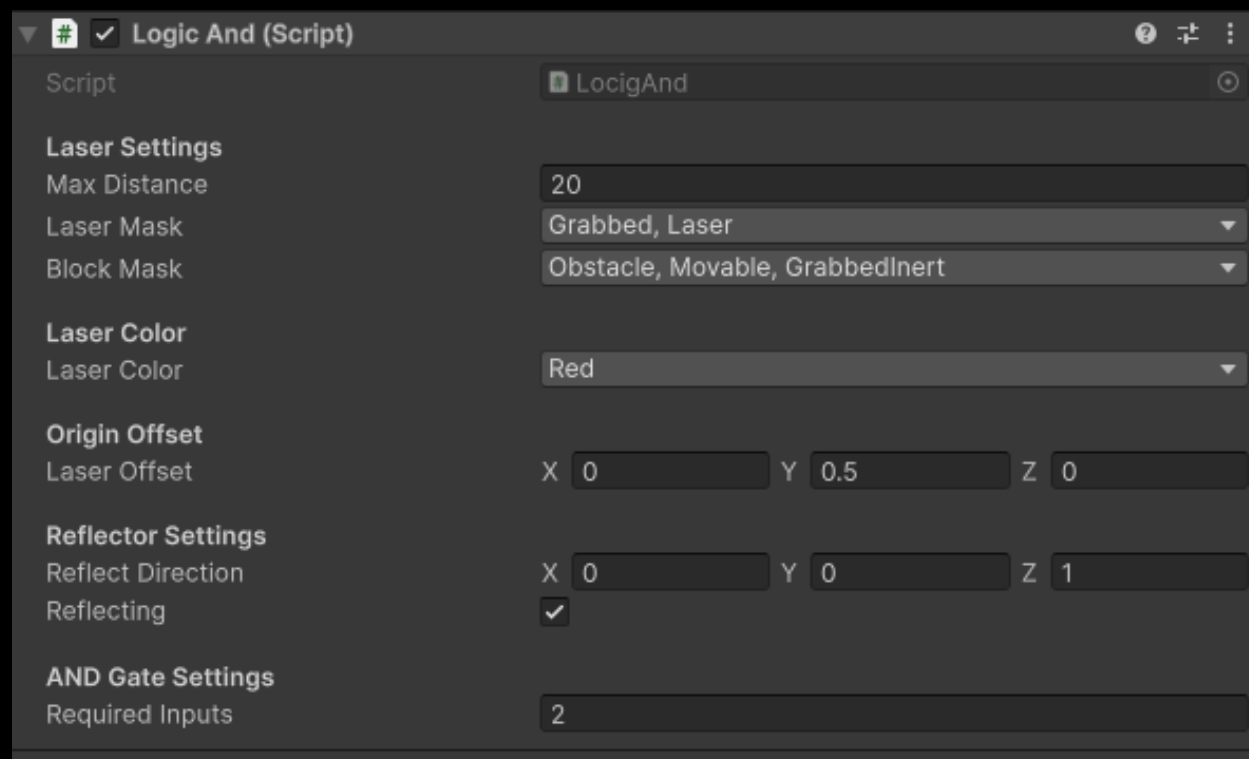
2 riferimenti
protected Vector3 CalculateOutputDirection(Vector3 hitPoint)
{
    if (reflecting)
    {
        return transform.TransformDirection(reflectDirection).normalized;
    }
    else
    {
        Vector3 localHit = transform.InverseTransformPoint(hitPoint);
        Vector3 localNormal = Vector3.zero;
        if (Mathf.Abs(localHit.x) > Mathf.Abs(localHit.z))
            localNormal = new Vector3(Mathf.Sign(localHit.x), 0, 0);
        else
            localNormal = new Vector3(0, 0, Mathf.Sign(localHit.z));
        return transform.TransformDirection(-localNormal).normalized;
    }
}
```



Instead **Reflector** controls the general reaction to being hit by a **laser**.

LASER

The **LogicGate** script defines the common behaviour of all the logic ports.



```
public override void OnLaserHit(Vector3 hitPoint, LaserColor incomingColor)
{
    if (incomingColor == LaserColor.Preview)
    {
        if (isRealHitThisFrame) return;
        isPreviewFrame = true;
    }
    else
    {
        isRealHitThisFrame = true;
    }
    isHitThisFrame = true;
    currentEmitDirection = CalculateOutputDirection(hitPoint);
}
```

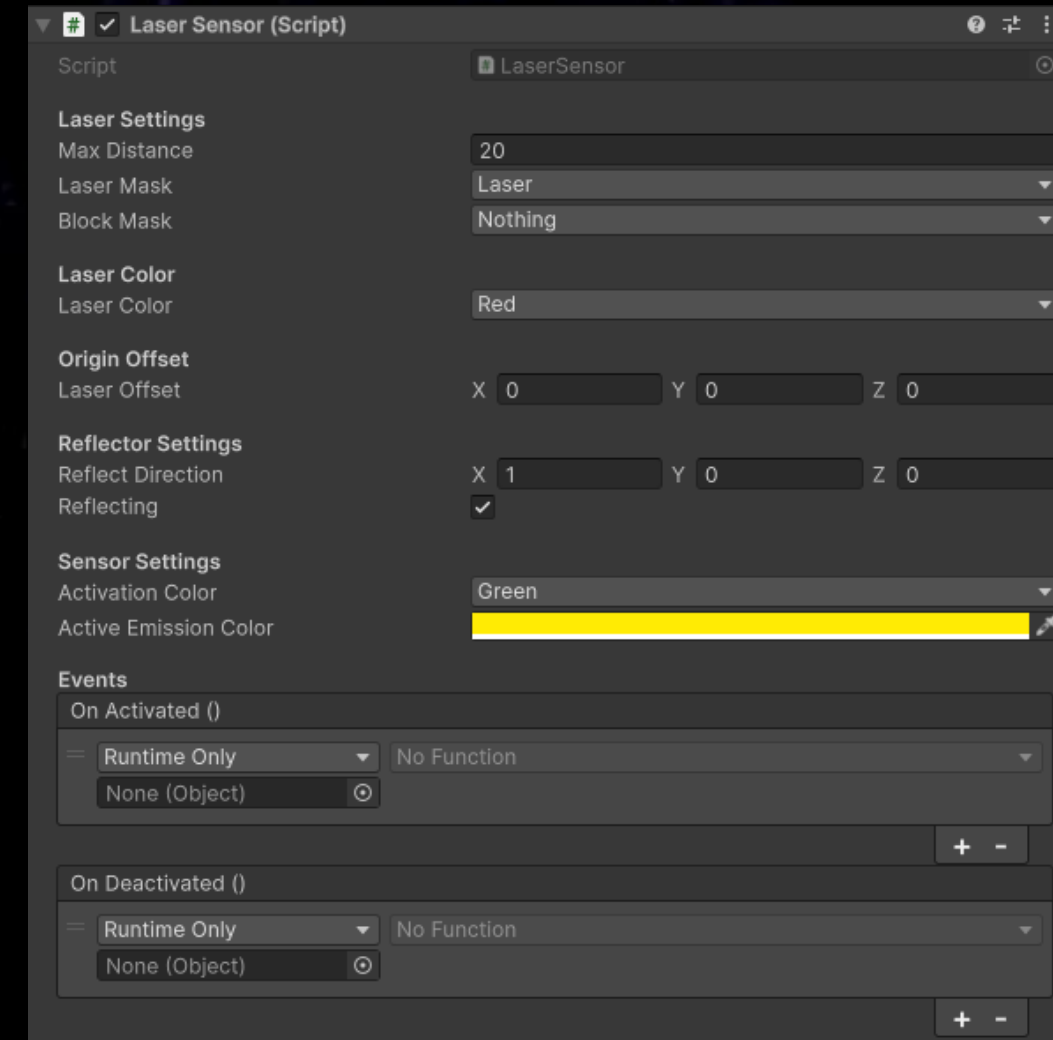
```
Messaggio Unity | 1 riferimento
protected override void LateUpdate()
{
    if (isHitThisFrame)
    {
        LaserColor colorToEmit = (isRealHitThisFrame || !isPreviewFrame)
            ? EvaluateLogic()
            : LaserColor.Preview;
        FireRay(LaserOrigin, currentEmitDirection, colorToEmit);
    }
    else
    {
        ClearLaser();
    }
    ResetGate();
    isRealHitThisFrame = false;
    isHitThisFrame = false;
    isPreviewFrame = false;
}
```

LASER

```
public override void OnLaserHit(Vector3 hitPoint, LaserColor incomingColor)
{
    isHitThisFrame = true;
    if (incomingColor == activationColor)
    {
        if (!isActive)
        {
            ActivateSensor();
        }
    }
    else
    {
        if (isActive)
        {
            DeactivateSensor();
        }
    }
}

1 riferimento
private void ActivateSensor()
{
    isActive = true;
    if (rend != null)
    {
        rend.material.EnableKeyword("_EMISSION");
        rend.material.SetColor("_EmissionColor", activeEmissionColor * 3f);
    }
    OnActivated?.Invoke();
}

2 riferimenti
private void DeactivateSensor()
{
    isActive = false;
    Illuminate(false);
    OnDeactivated?.Invoke();
}
```



The **sensor** reacts to **lasers** of a specific **color**.

LASER

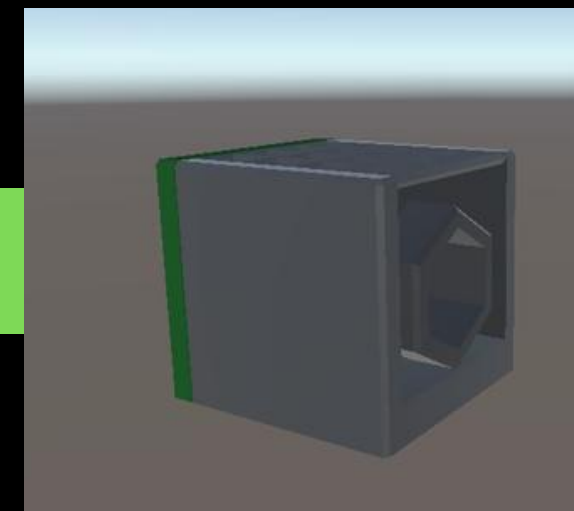
There are multiple object used in the game that allows this system to work:

- **Emitter**: Creates the **laser** (both in preview and in active mode). **LaserInteractable** allows it to be activated by the **player**.
- **Sensor**: Receives the **signal** and uses **Unity events**.



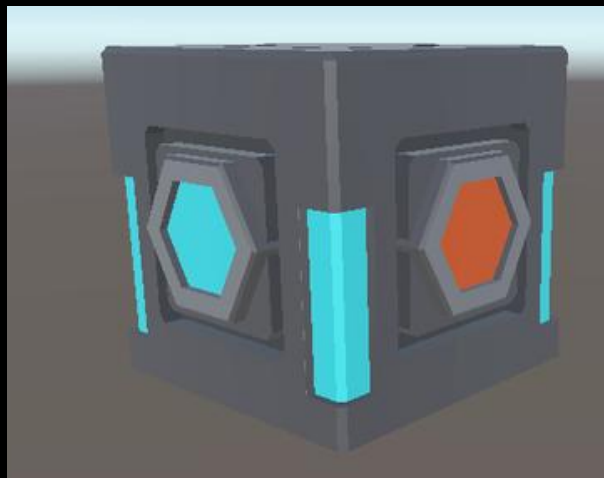
LASER SENSOR

LASER EMITTER



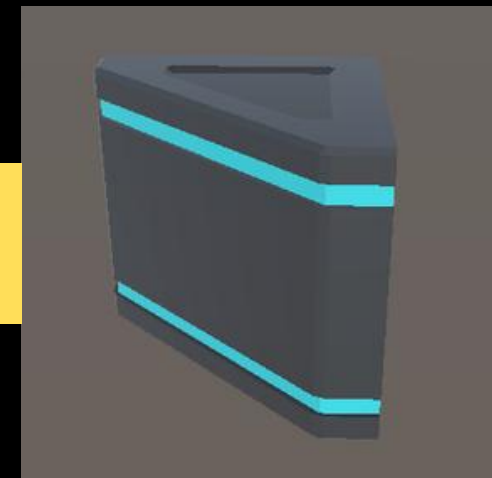
LASER

- **Reflector**: Generic block that **reflects** and can change the trajectory of the **laser**. It hasn't been used in the final game.
- **Or** / **And**: Evaluates the **input** based on **And/Or** logic.
- **Not**: If the **input** is active it **inverts** its value.

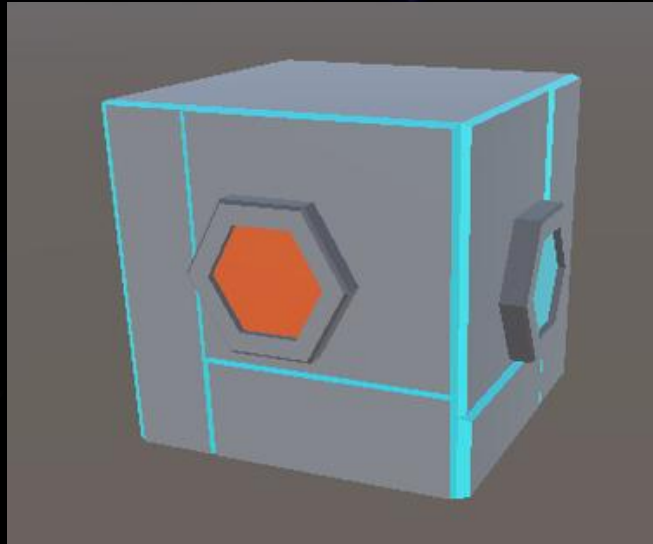


LOGIC NOT

LASER REFLECTOR

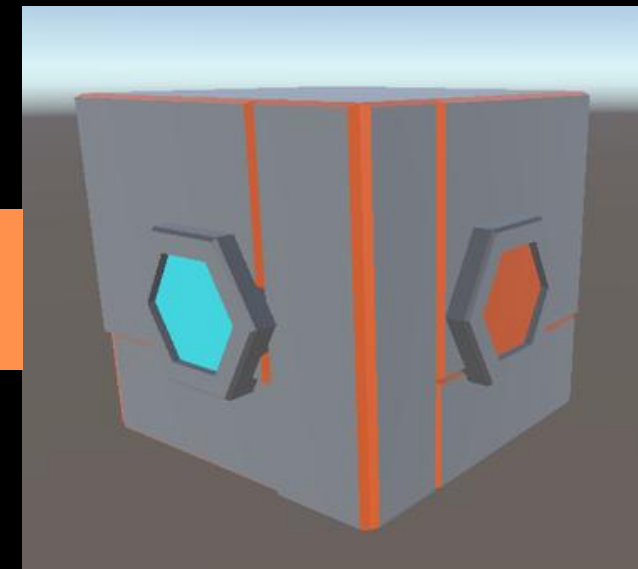


LASER



LOGIC OR

LOGIC AND



LASER

The different **logic ports** react differently to the various **laser colors**.

A	NOT
GREEN	RED
RED	GREEN
GRAY	GRAY

Figura 1: NOT Truth Table

A	B	AND
GREEN	GREEN	GREEN
GREEN	RED	RED
RED	GREEN	RED
RED	RED	RED
RED	GRAY	GRAY
GREEN	GRAY	GRAY
GRAY	RED	GRAY
GRAY	GREEN	GRAY
GRAY	GRAY	GRAY

Figura 2: AND Truth Table

A	B	OR
GREEN	GREEN	GREEN
GREEN	RED	GREEN
RED	GREEN	GREEN
RED	RED	RED
RED	GRAY	GRAY
GREEN	GRAY	GRAY
GRAY	RED	GRAY
GRAY	GREEN	GRAY
GRAY	GRAY	GRAY

Figura 3: OR Truth Table

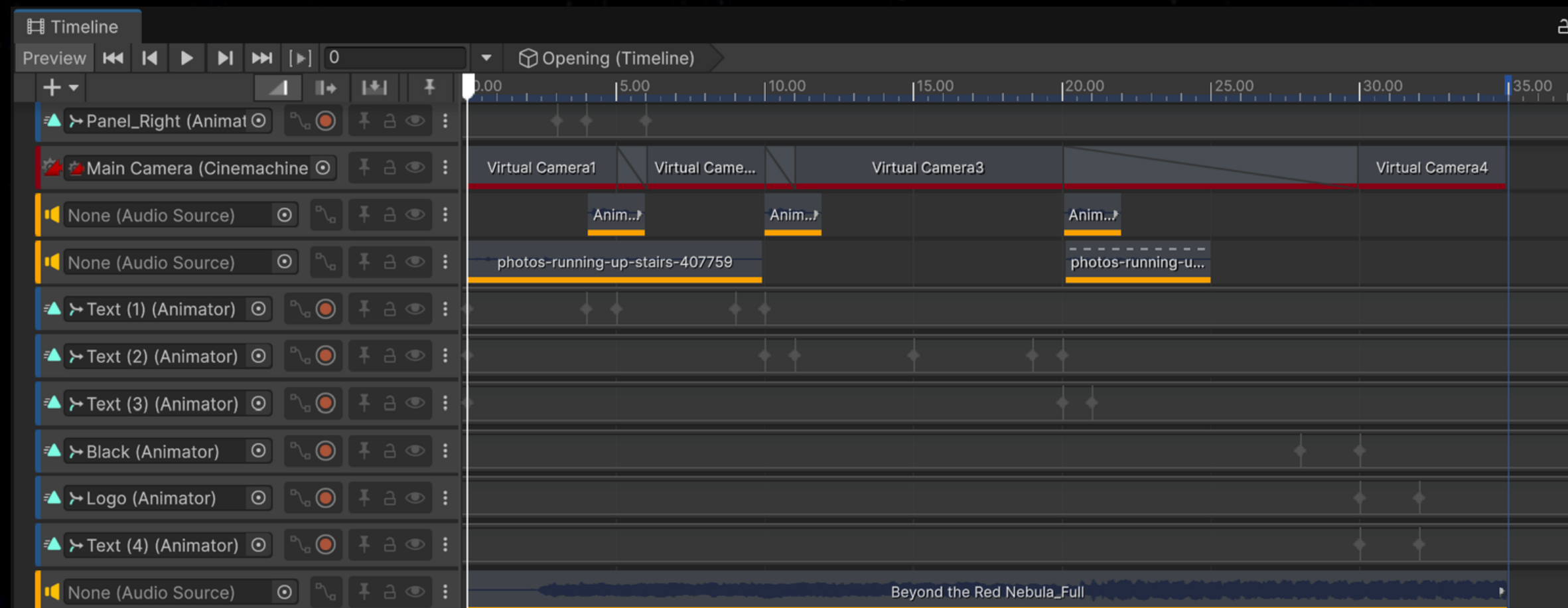
TOPIC 4

CUTSCENES MANAGEMENT
AND NPC INTERACTION

TIMELINE AND NPC INTERACTABLE SCRIPT

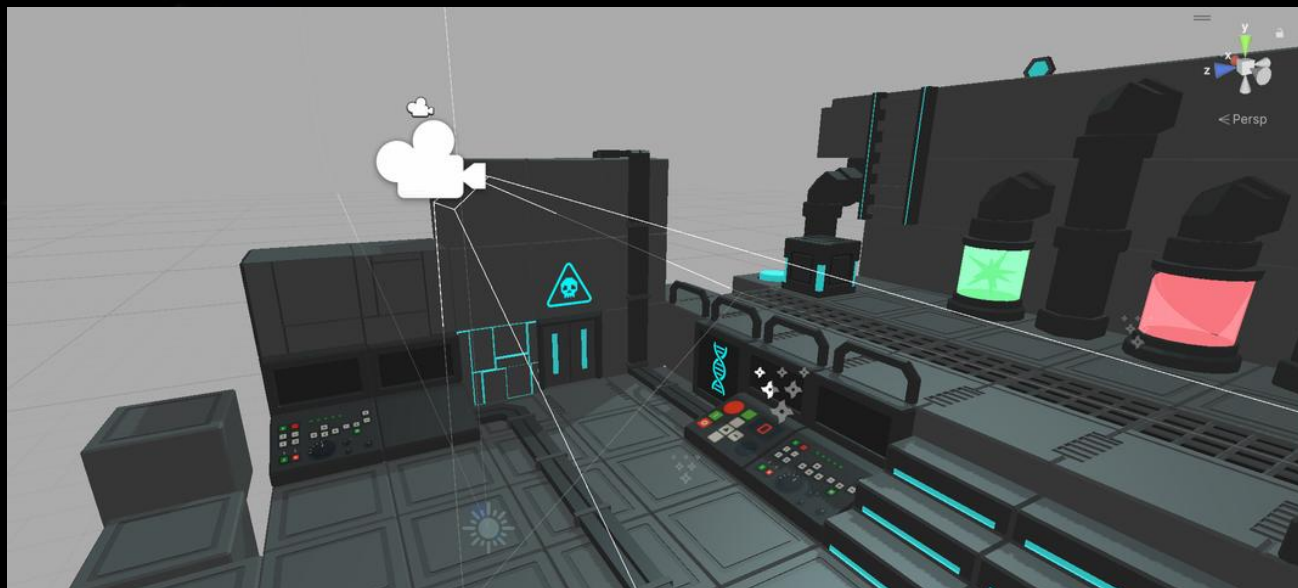
MAINS CUTSCENES

In Unity, cutscenes are managed via **timelines**, which allow you to control the position of objects and virtual cameras. They also allow you to apply music and sounds.

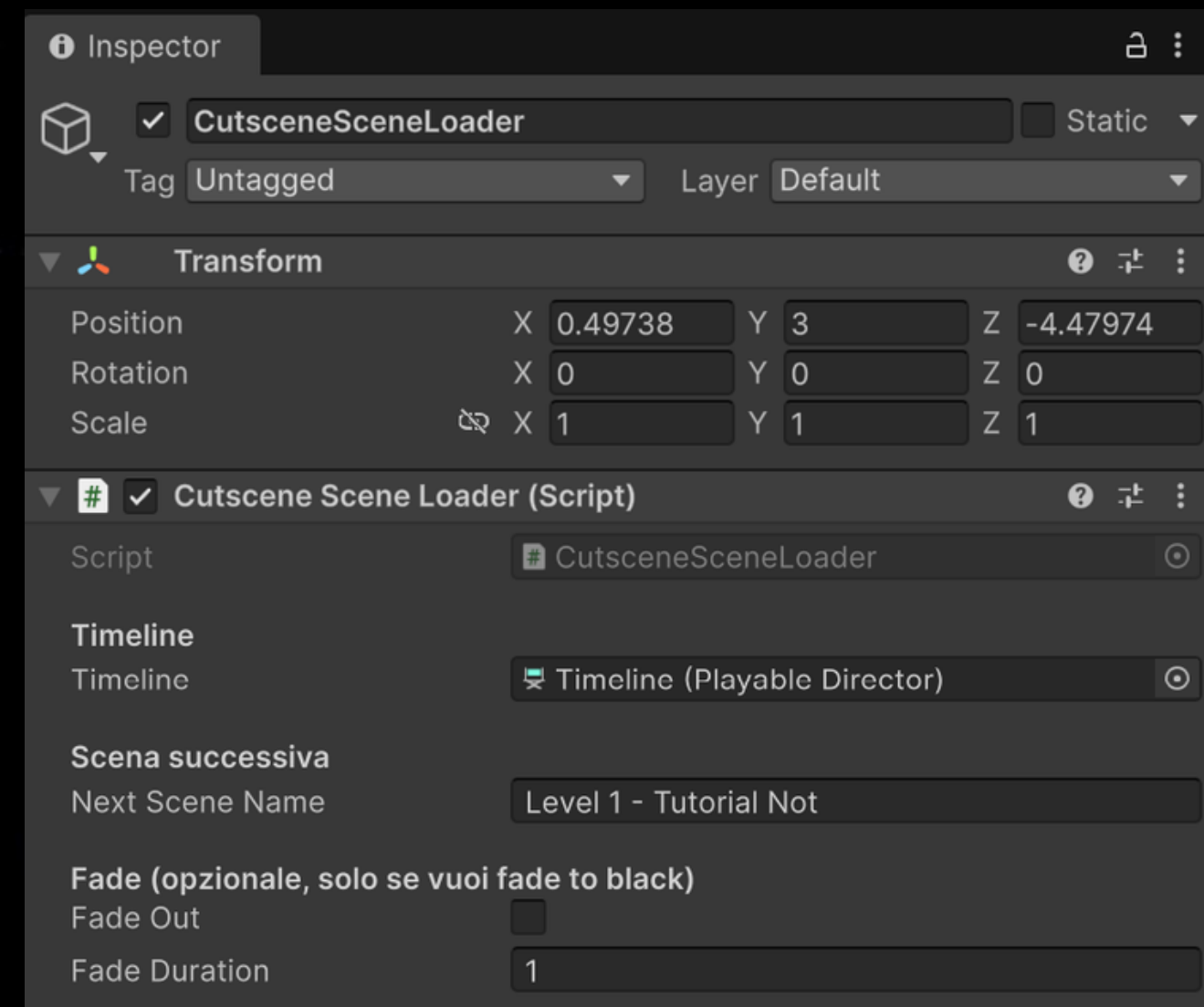


MAINS CUTSCENES

The cutscene is located in a **separate scene**, loaded when the game launches.

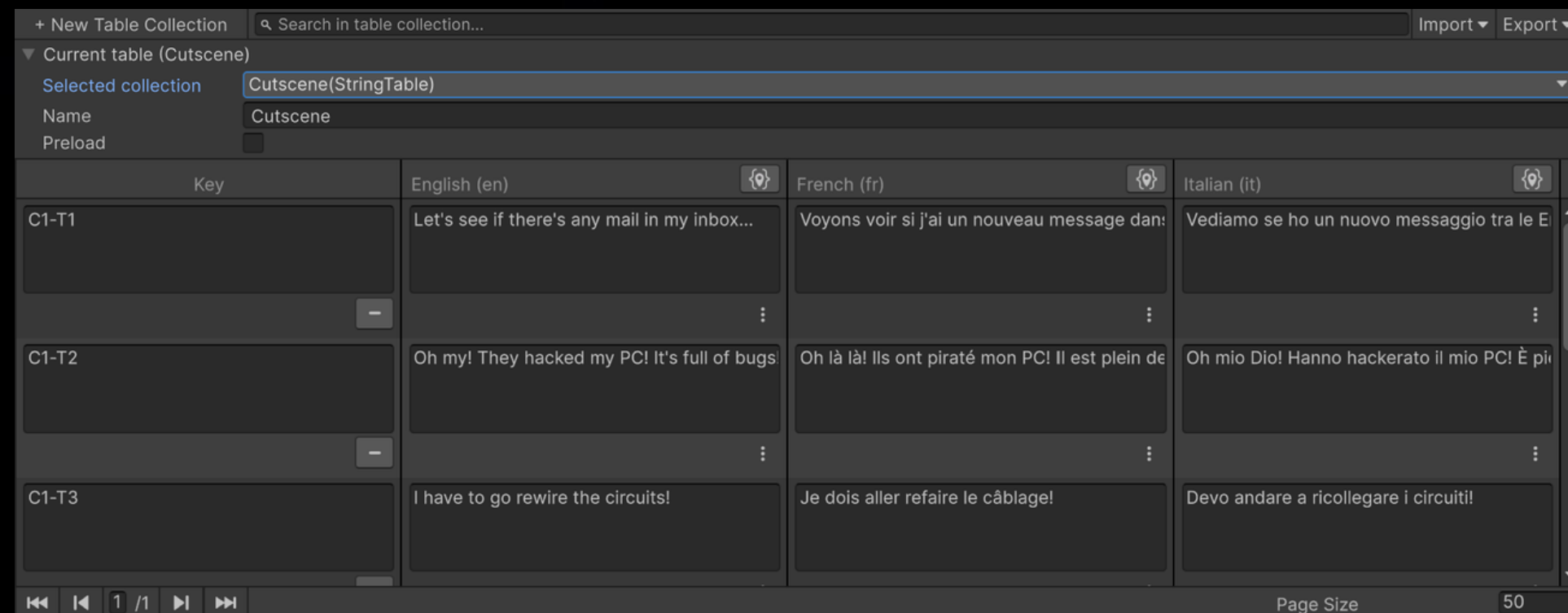


Using scripts, we can then decide which **scene** (i.e., which level) will load at the end of the timeline, as well as how long the **fade** will last.

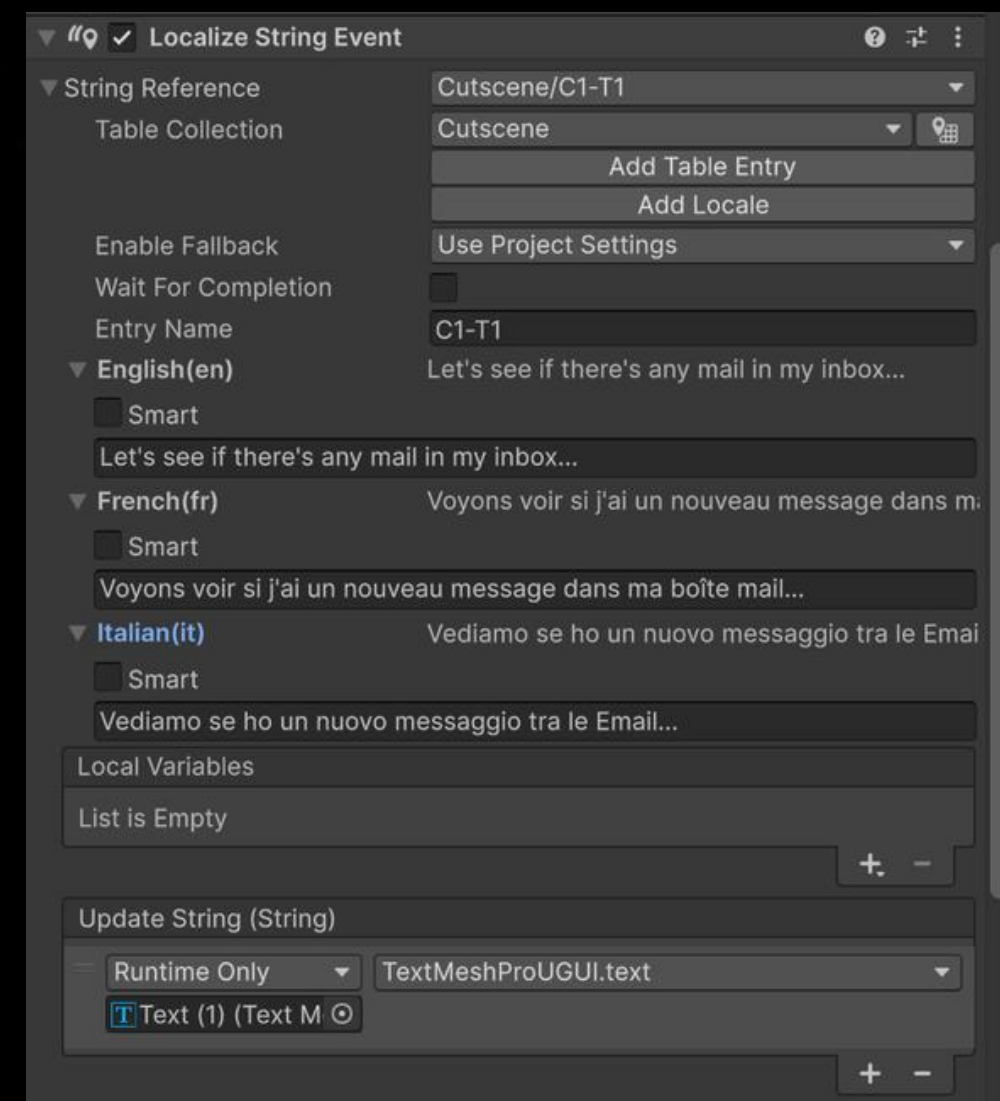


MAINS CUTSCENES

Using the **Collection Tables**, we can then **translate** the texts in the cutscenes.

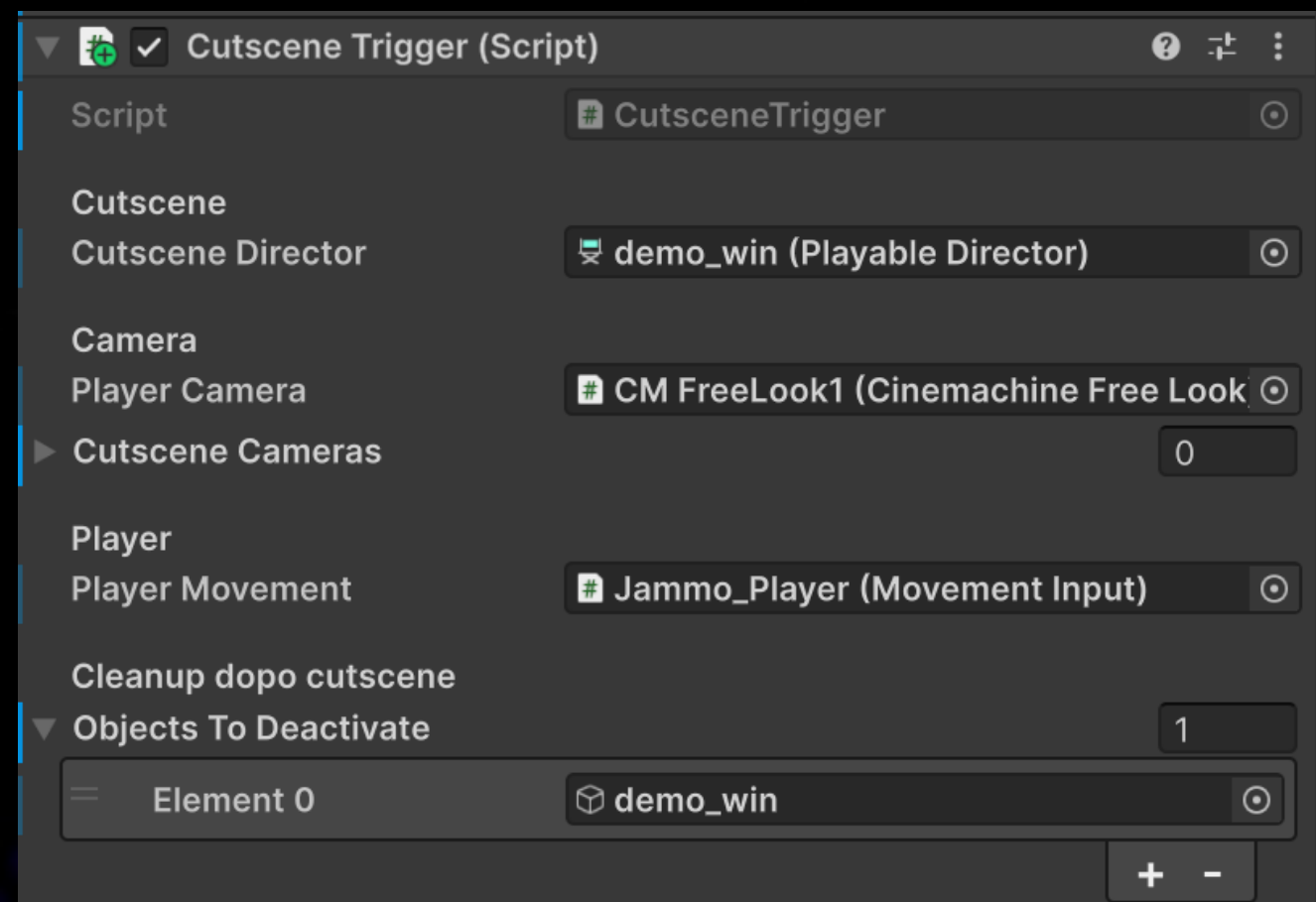
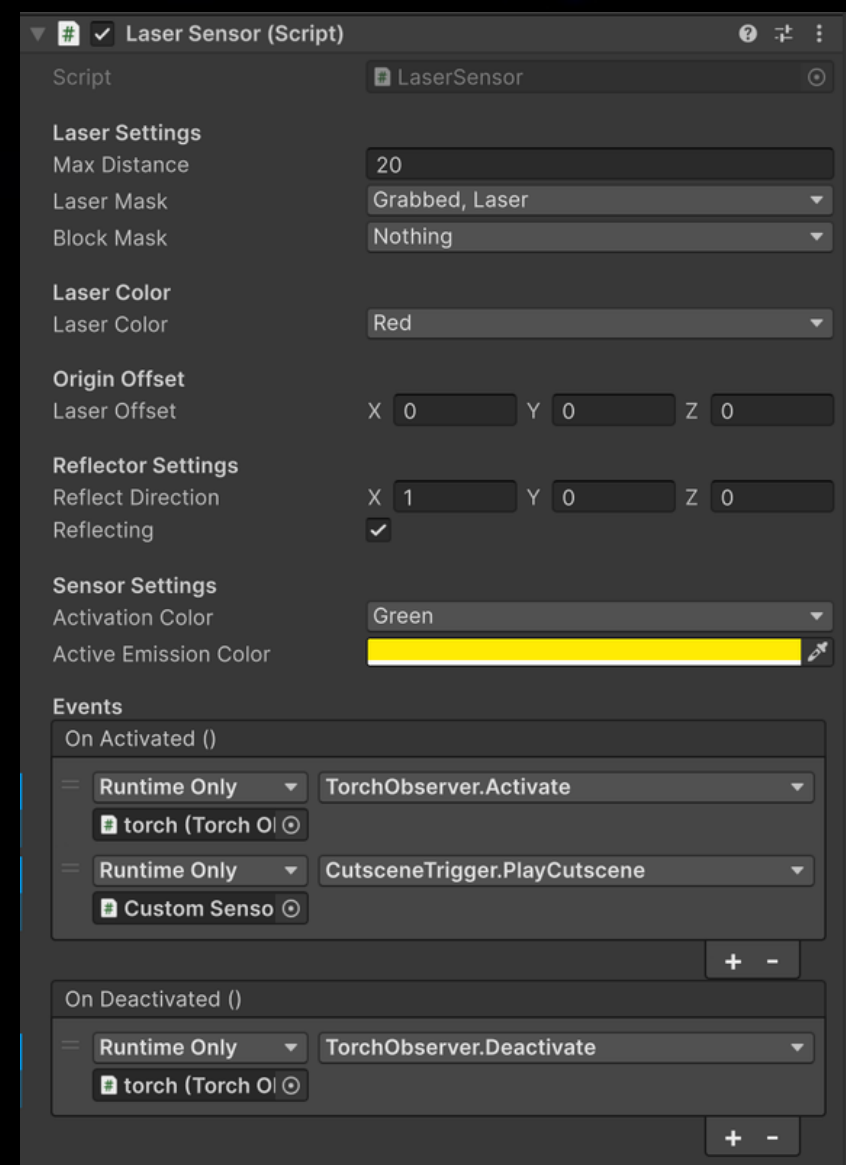


In our case, we limited ourselves to Italian, French, and English. But these could easily be expanded.



LEVELS CUTSCENES

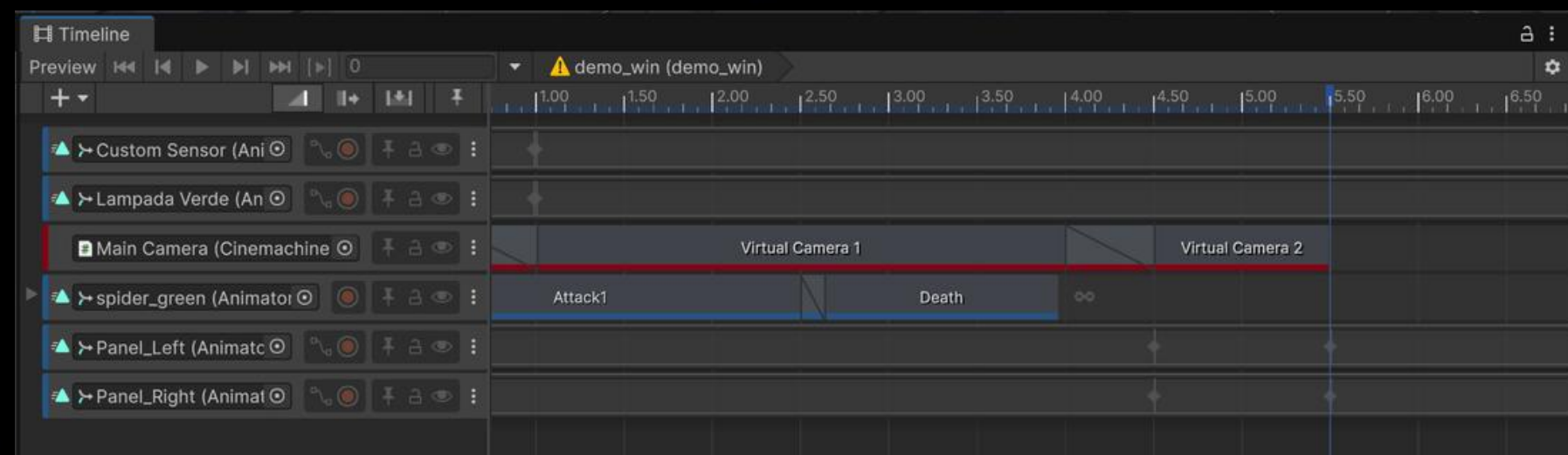
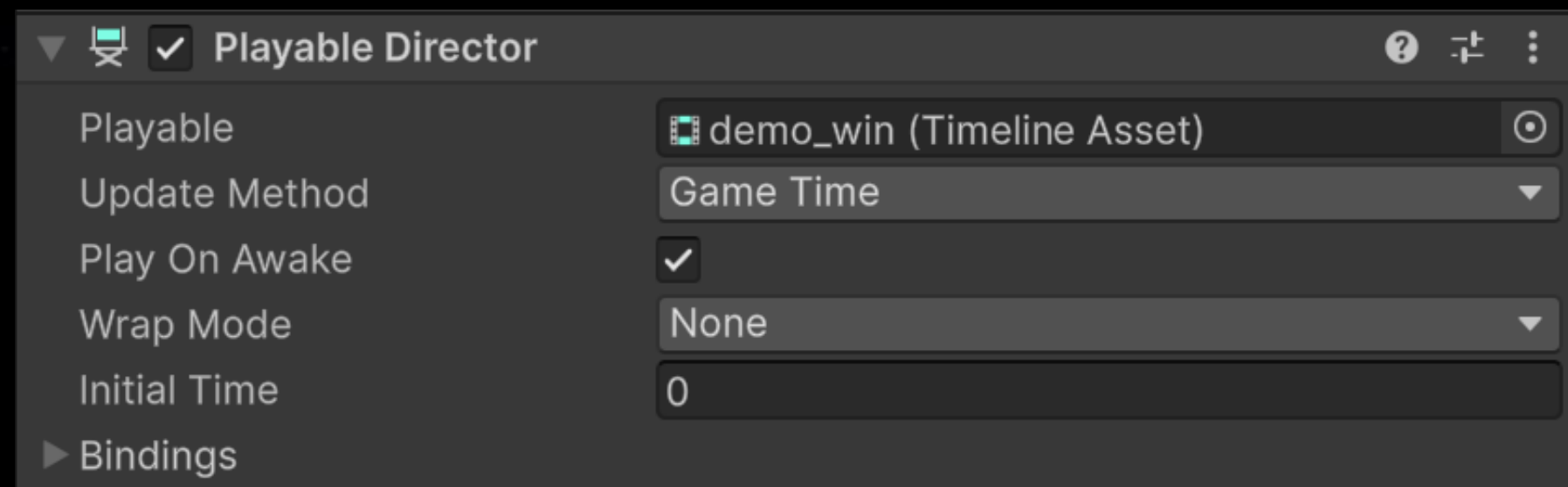
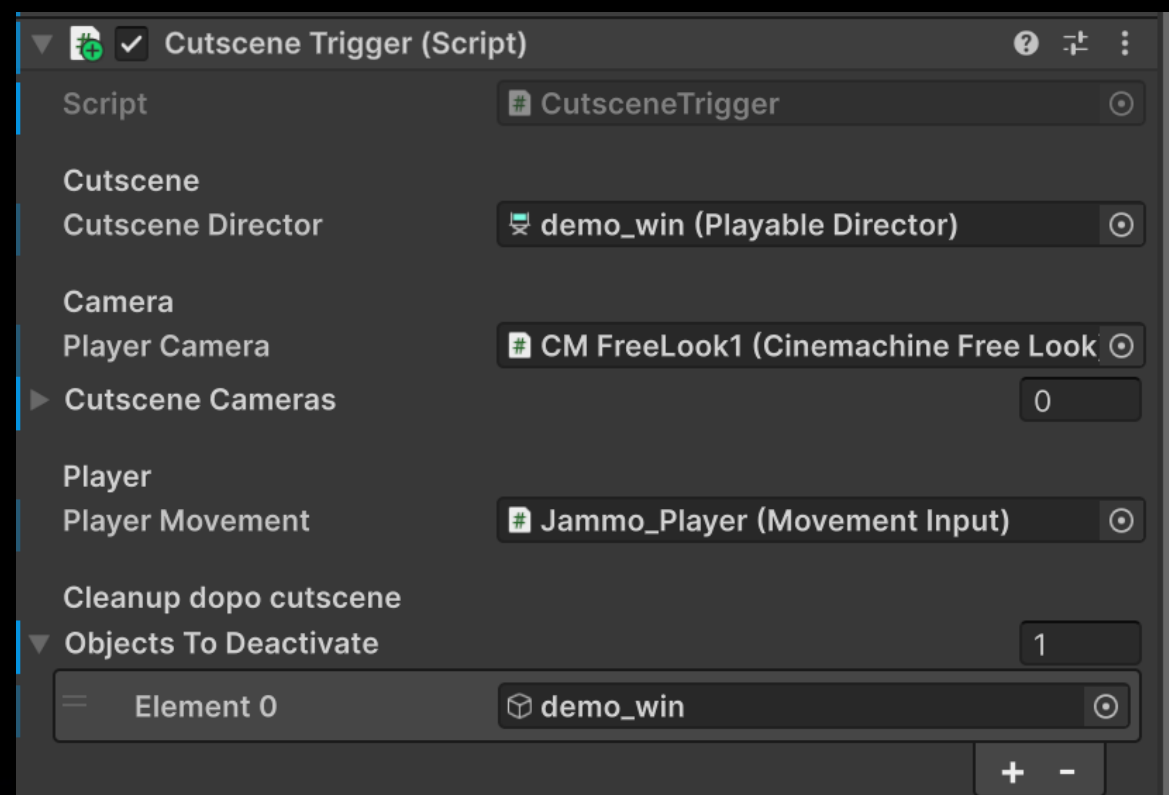
To launch a cutscene in the middle of a level, we first wait for **LaserSensor** to call an event. This is called a **CutsceneTrigger**, which immobilizes the player.



LEVELS CUTSCENES

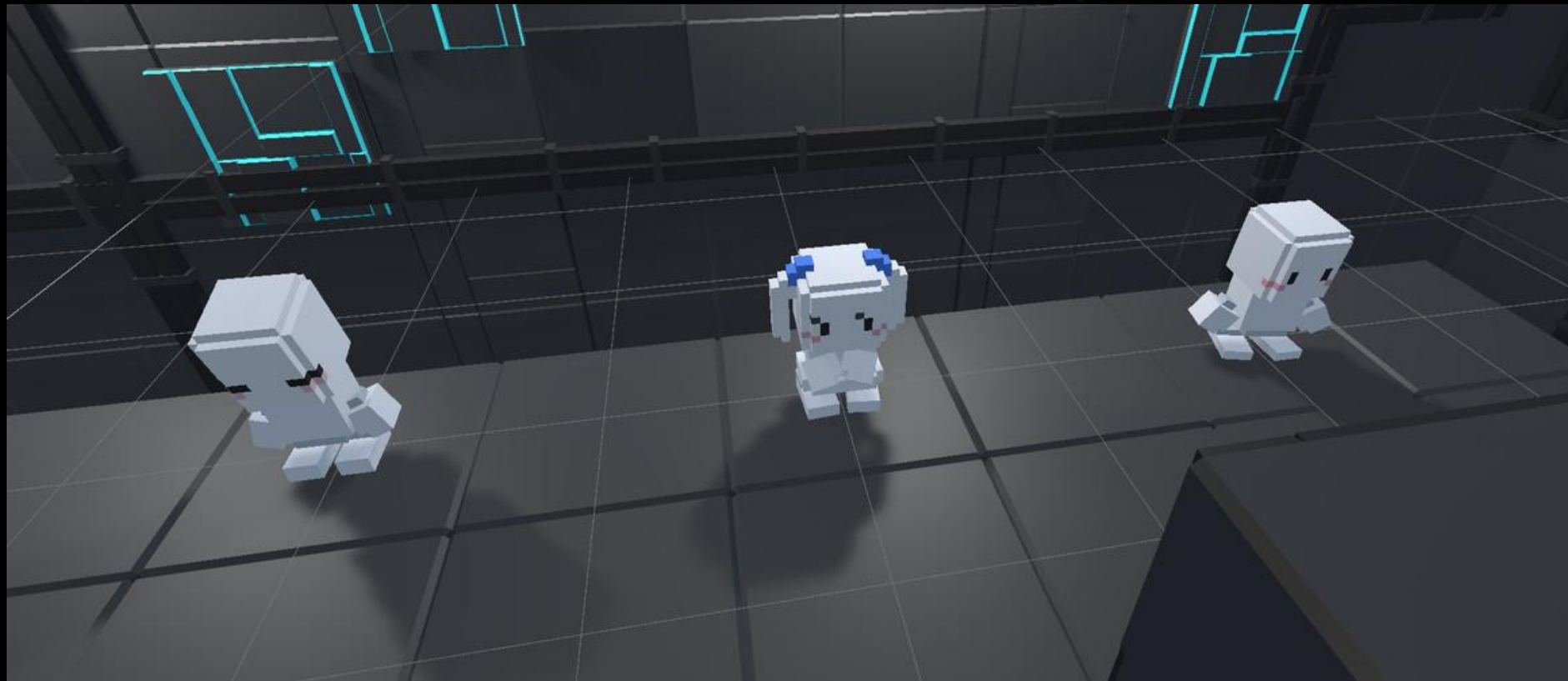
The cutscene is then started by the **Playable Director**. At the end of the cutscene, **CutsceneTrigger** returns the movement to the player.

The cutscene is deactivated and control is returned to the player's camera



NPC INTERACTION

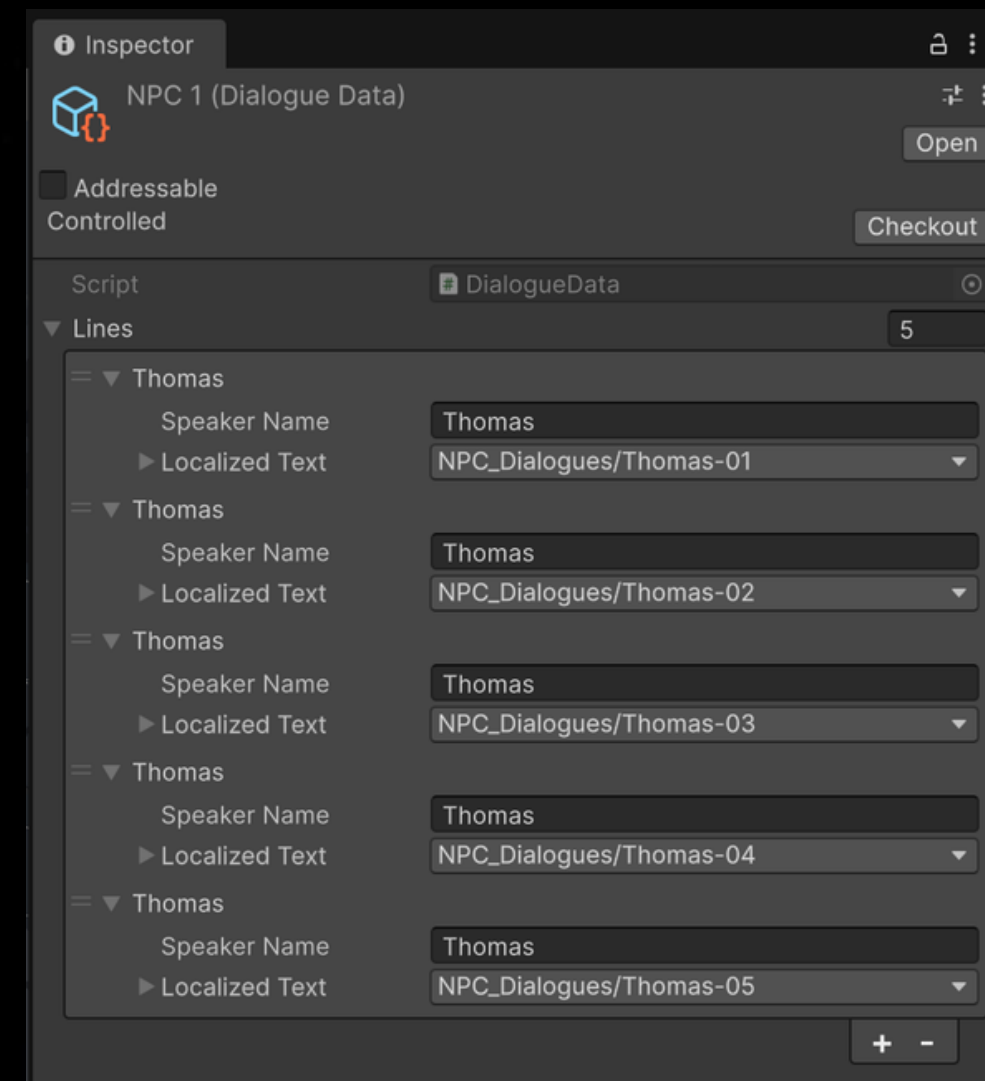
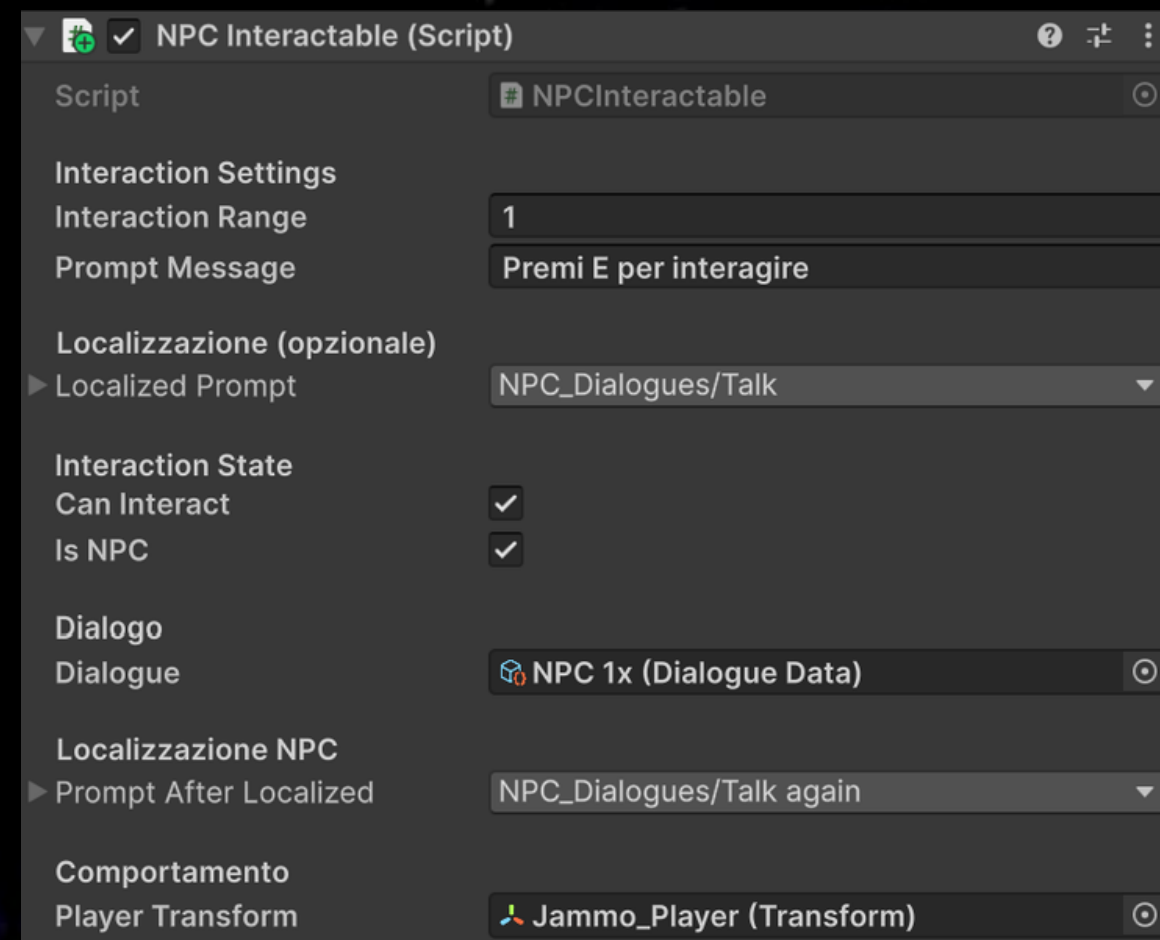
NPCs have a small script called `NPCinteractable`, a subclass of `Interactable`.



NPC INTERACTION

The script does three things:

- Makes the NPC turn towards the player.
- Immobilizes the player for the duration of the conversation.
- Plays the text contained in the DialogueData.



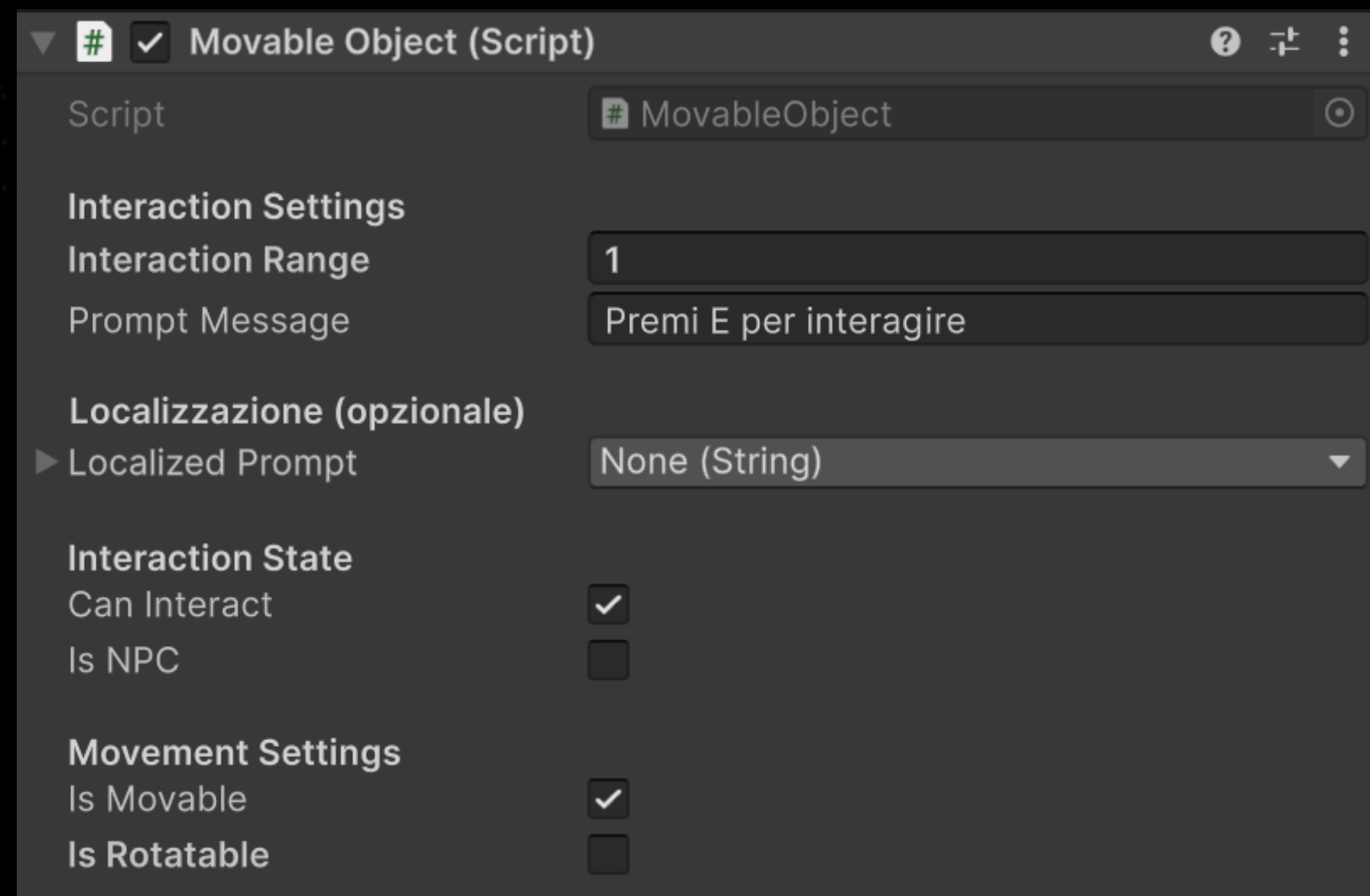
TOPIC 5

FUTURE EXPANSIONS

FUTURE WAY TO EXPAND THE GAME

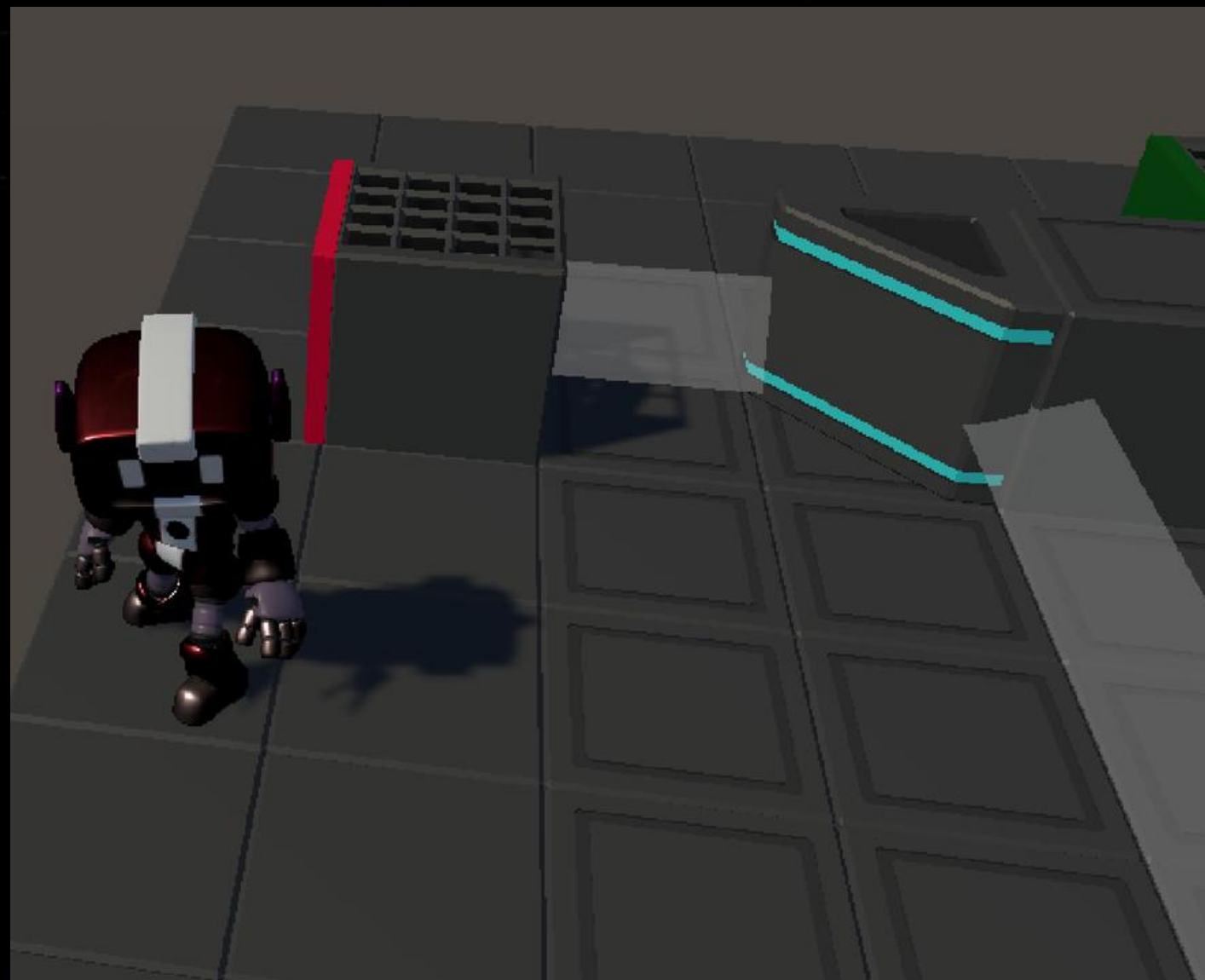
ROTATE BLOCKS

The game is already programmed to allow blocks to **rotate**, thus allowing them to reflect the beam in other directions.



REFLECT

The game features a **block that reflects** rays. We didn't use it in the levels because we already had enough material and lacked a model to use for the block.



In our tests, we used a flipped ramp. But rotating the block interferes with the positioning of the beam.

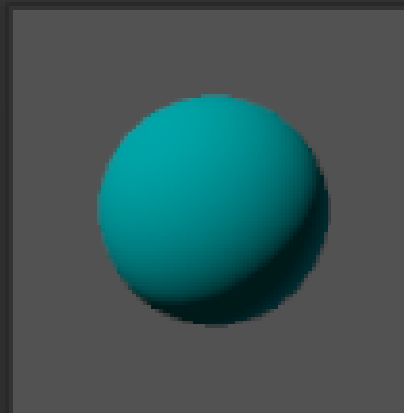
GRID TILES

Possible future improvements would be adding **new types of tile**, this would be easy to do as the system is modular so creating a new script extending GridTileBase would be enough.

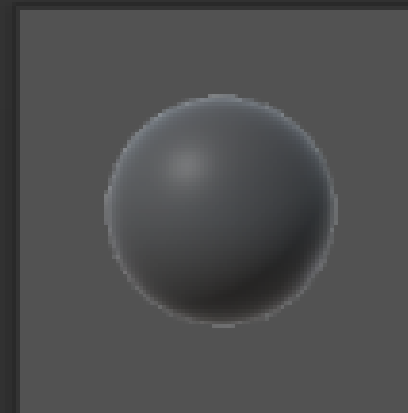
Assets > Prefabs > Grid point



 bounce



 ice

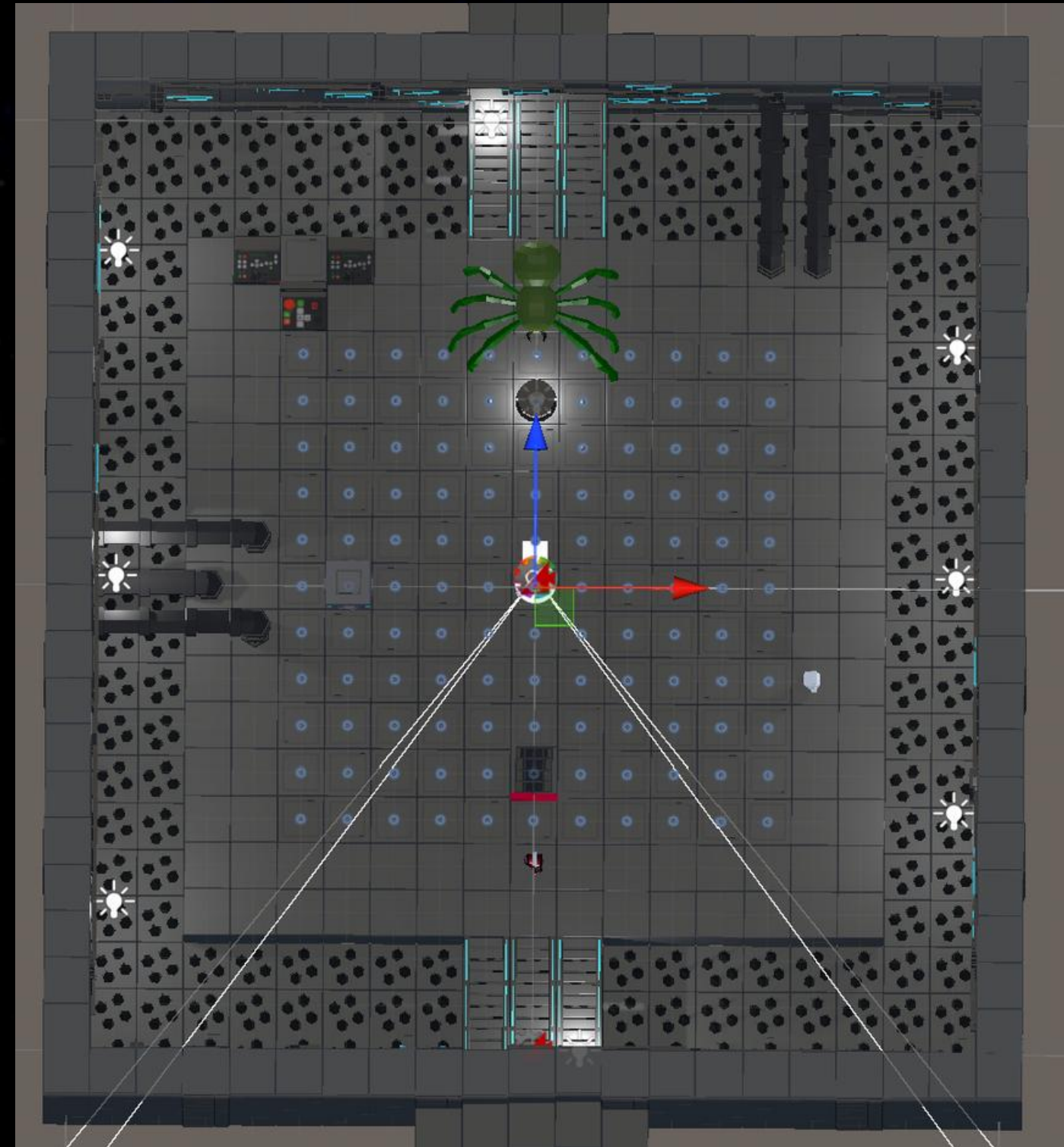


 point

GRID TILES

Moreover the grid system like now creates a default grid that can be personalized in the code or later manually in the editor tile by tile.

An improvement would be to allow to customize the default grid by the editor through some parameters.





GAME DEMO

GAME DEMO

NOTANDOR

New Game

Load Game

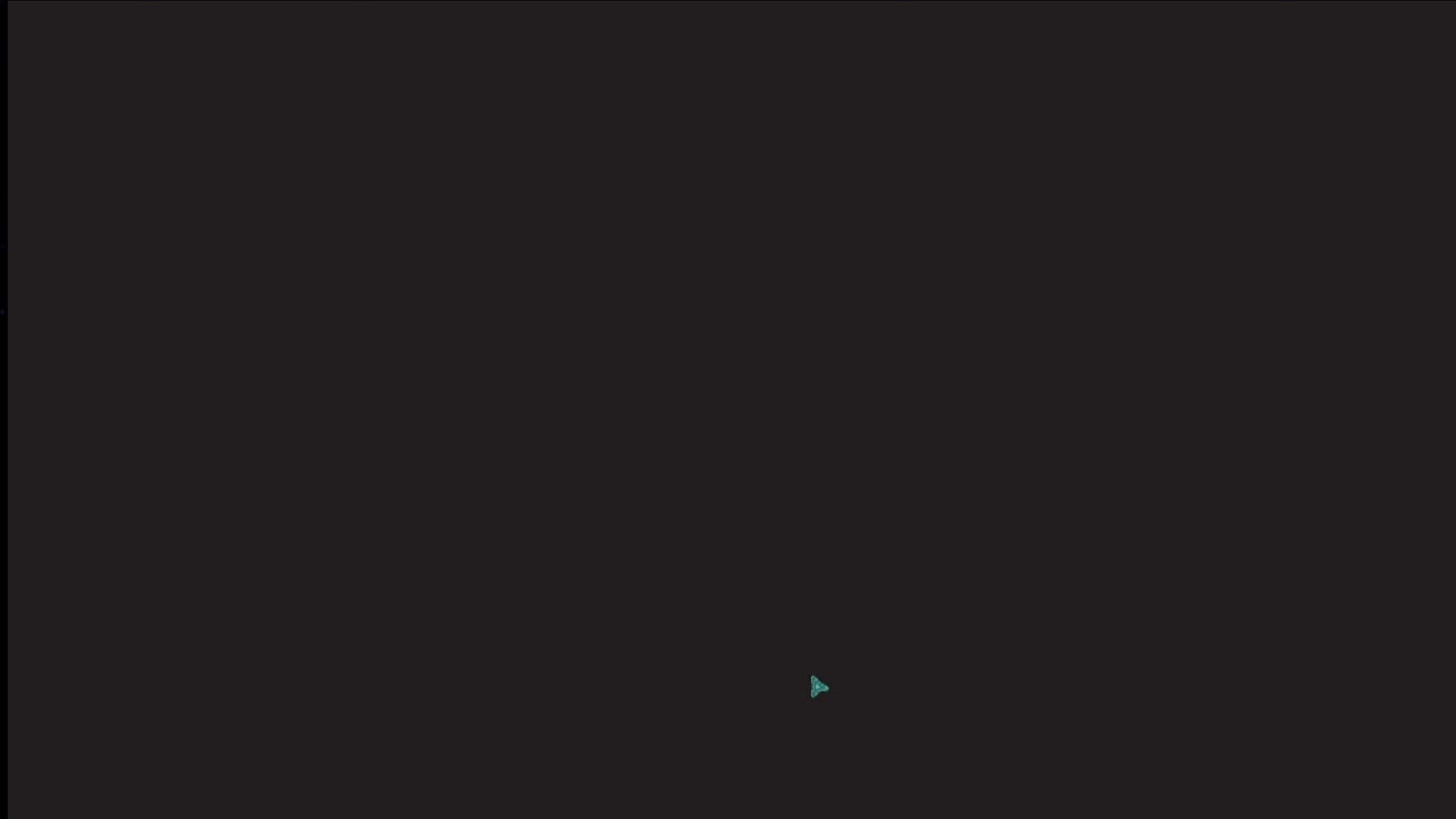
Options

Quit Game



Premi Alt+Z per aprire la
sovrapposizione NVIDIA

GAME DEMO - QUALITY



THANKS FOR PLAYING!

[SYSTEM LOG]

Computer Status : NORMAL

Diagnostics : COMPLETE

Error Report : NONE

Bug Scan : COMPLETE

Result : All bugs resolved
